

Unit 07: The Shell and popular Editors

CONTENTS
Objectives
Introduction
7.1 The Shell
7.2 Standard Input and Standard Output
7.3 Redirection
7.4 Pipes
7.5 Filters
7.6 Running a Command in the Background
7.7 Filename Generation/Pathname Expansion
7.8 Builtins
7.9 vim
7.10 Modes in vim
7.11 Introduction to vim Features
7.12 Command Mode
7.13 Input Mode
7.14 Emacs
Summary
Keywords
Self Assessment
Answers for Self Assessment
Review Questions:
Further Readings

Objectives

After studying this unit, you will be able to:

- Understand the Command Line
- Understand the Standard Input and Standard Output
- Understand about filename generation and pathname extension
- Understand about builtins
- Know how to use and features of vim
- Understand the command mode and input mode
- Know the difference between emacs and vim
- Understand the functionalities and basic editing commands in emacs

Introduction

The important component of any computer system is an operating system. An operating system is the software that directly manages a system's hardware and resources, like CPU, memory, and storage. The OS sits between applications and hardware and makes the connections between

Linux and Shell Scripting

all your software and the physical resources that do the work. Think about an OS like a car engine. An engine can run on its own, but it becomes a functional car when it's connected with a transmission, axles, and wheels. Without the engine running properly, the rest of the car won't work. An Operating system is made of many components, but its two prime components are – kernel and shell.

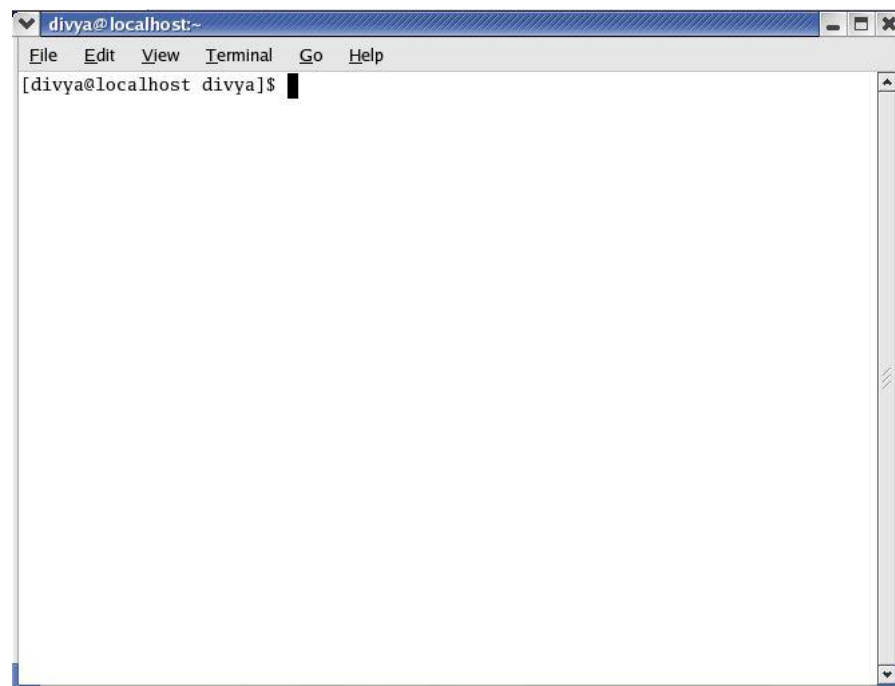
A Kernel is at the nucleus of a computer. It makes the communication between the hardware and software possible. While the Kernel is the innermost part of an operating system, a shell is the outermost one. It is the base component of the OS. Without it, the OS doesn't work. The kernel manages the system's resources and communicates with the hardware. It's responsible for memory, process, and file management. A shell in a Linux operating system takes input from you in the form of commands, processes it, and then gives an output. It is the interface through which a user works on the programs, commands, and scripts. A shell is accessed by a terminal which runs it. When you run the terminal, the Shell issues a command prompt (usually \$), where you can type your input, which is then executed when you hit the Enter key. The output or the result is thereafter displayed on the terminal. The Shell wraps around the delicate interior of an OS protecting it from accidental damage. Hence the name Shell.

7.1 The Shell

A Shell provides you with an interface to the Unix system. It gathers input from you and executes programs based on that input. When a program finish executing, it displays that program's output. Shell is an environment in which we can run our commands, programs, and shell scripts. There are different flavors of a shell, just as there are different flavors of operating systems. Each flavor of shell has its own set of recognized commands and functions. You can cause the shell to execute various types of programs – such as shell scripts, application programs, and programs you have written – in the same way. The line that contains the command, including any arguments, is called the command line. The syntax for a basic command line is

command [arg1] [arg2] ... [argn] RETURN

The prompt, \$, which is called the command prompt, is issued by the shell. While the prompt is displayed, you can type a command. Shell reads your input after you press Enter



Rules for commands

There are some rules for writing the commands. These are:

- One or more SPACES must separate elements on the command line.
- The command is the name of the command, arg1 through argn are arguments, and RETURN is the keystroke that terminates all command lines.
- The brackets in the command-line syntax indicate that the arguments they enclose are optional.
- Not all commands require arguments.

Command Name

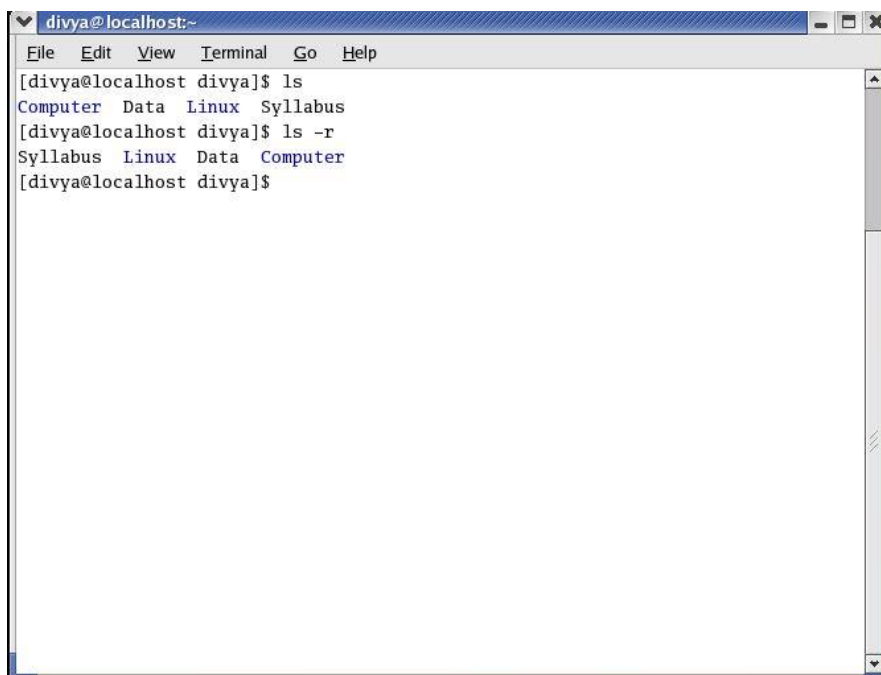
Some useful Linux command lines consist of only the name of the command without any arguments. Commands that require arguments typically give a short error message, called a usage message.

Arguments

On the command line each sequence of nonblank characters is called a token or word. An argument is a token, such as a filename, string of text, number, or other object that a command acts on. For example, the argument to a vim or emacs command is the name of the file you want to edit. The following command line shows cp copying the file named temp to temp copy: `$ cp temp temp copy`. Arguments are numbered starting with the command itself, which is argument zero. cp argument-0, temp argument-1 and temp copy argument-2. The cp utility requires at least two arguments on the command line.

Options

An option is an argument that modifies the effects of a command. You can frequently specify more than one option, modifying the command in several different ways. Most utilities require you to prefix options with a single hyphen. However, this requirement is specific to the utility and not the shell. GNU program options are frequently preceded by two hyphens in a row. For example, `--help` generates a (sometimes extensive) usage message.



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ ls  
Computer Data Linux Syllabus  
[divya@localhost divya]$ ls -r  
Syllabus Linux Data Computer  
[divya@localhost divya]$
```

Combining options

When you need to use several options, you can usually group multiple single-letter options into one argument that starts with a single hyphen; do not put SPACES between the options. You cannot combine options that are preceded by two hyphens in this way. Specific rules for combining options depend on the program you are running.

Option arguments

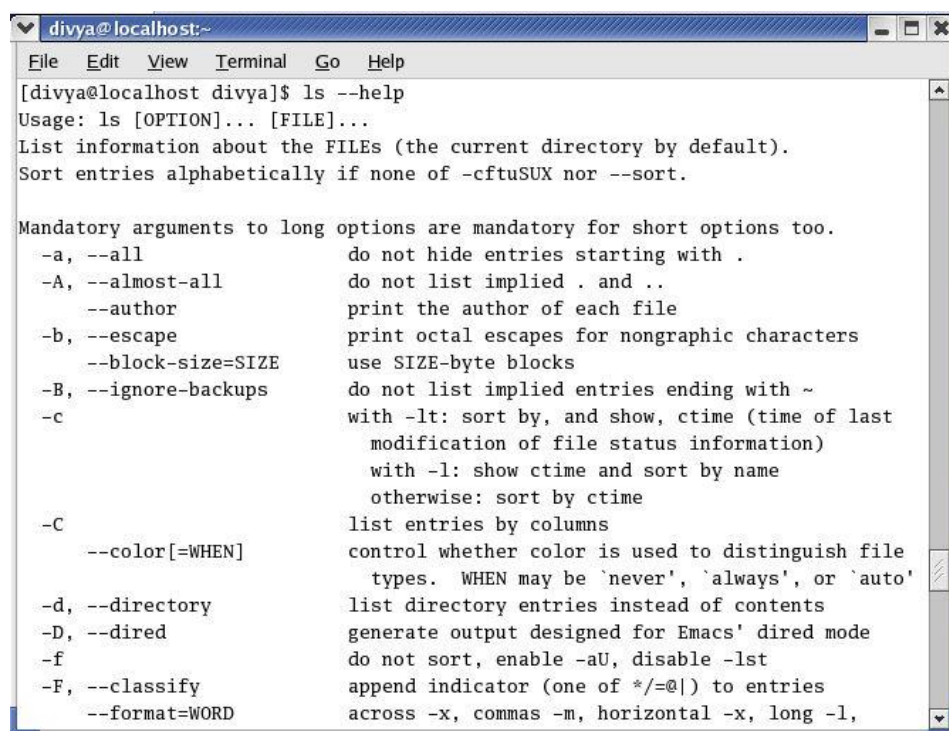
Some utilities have options that themselves require arguments. For example, the gcc utility has a `-o` option that must be followed by the name you want to give the executable file that gcc generates. Typically, an argument to an option is separated from its option letter by a SPACE.

Arguments that start with a hyphen

Another convention allows utilities to work with arguments, such as filenames, that start with a hyphen. If a file's name is `-l`, the following command is ambiguous:

```
$ ls -l
```

This command could mean you want `ls` to display a long listing of all files in the working directory or a listing of the file named `-l`.



```
divya@localhost:~
File Edit View Terminal Go Help
[divya@localhost divya]$ ls --help
Usage: ls [OPTION]... [FILE]...
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of -cftuSUX nor --sort.

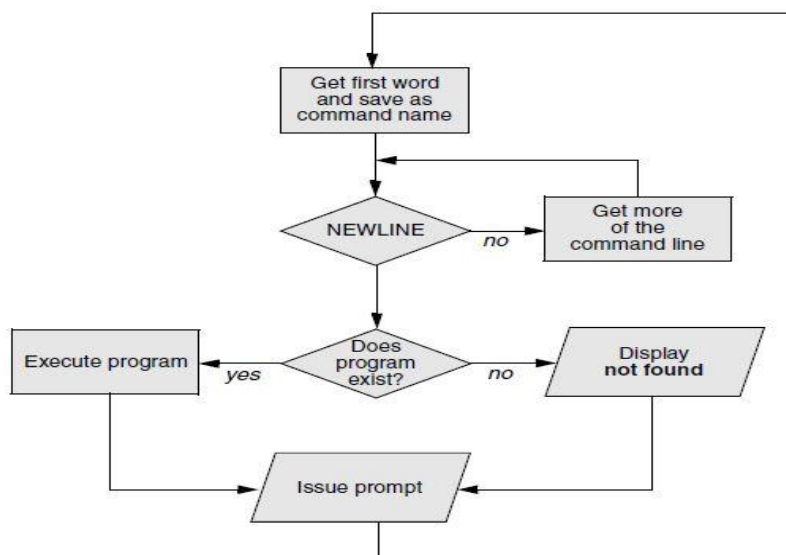
Mandatory arguments to long options are mandatory for short options too.
-a, --all                do not hide entries starting with .
-A, --almost-all        do not list implied . and ..
--author                print the author of each file
-b, --escape             print octal escapes for nongraphic characters
--block-size=SIZE       use SIZE-byte blocks
-B, --ignore-backups     do not list implied entries ending with ~
-c                      with -lt: sort by, and show, ctime (time of last
                        modification of file status information)
                        with -l: show ctime and sort by name
                        otherwise: sort by ctime
-C                      list entries by columns
--color[=WHEN]          control whether color is used to distinguish file
                        types. WHEN may be 'never', 'always', or 'auto'
-d, --directory         list directory entries instead of contents
-D, --dired              generate output designed for Emacs' dired mode
-f                      do not sort, enable -aU, disable -lst
-F, --classify          append indicator (one of */=@|) to entries
--format=WORD           across -x, commas -m, horizontal -x, long -l,
```

Processing the Command Line

As you enter a command line, the Linux tty device driver (part of the Linux kernel) examines each character to see whether it must take immediate action.

Key	Result
CONTROL-H	To erase a character
CONTROL-U	To kill a line
CONTROL-W	To erase a word

When the character you entered does not require immediate action, the device driver stores the character in a buffer and waits for additional characters. When you press RETURN, the device driver passes the command line to the shell for processing.



Parsing the command line

When the shell processes a command line, it looks at the line as a whole and parses (breaks) it into its component parts. Next, the shell looks for the name of the command. Usually, the name of the command is the first item on the command line after the prompt (argument zero). The shell typically takes the first characters on the command line up to the first blank (TAB or SPACE) and then looks for a command with that name. The command name (the first token) can be specified on the command line either as a simple filename or as a pathname. For example, you can call the ls command in either of the following ways:

```
$ ls
```

```
$ /bin/ls
```

Executing the Command Line

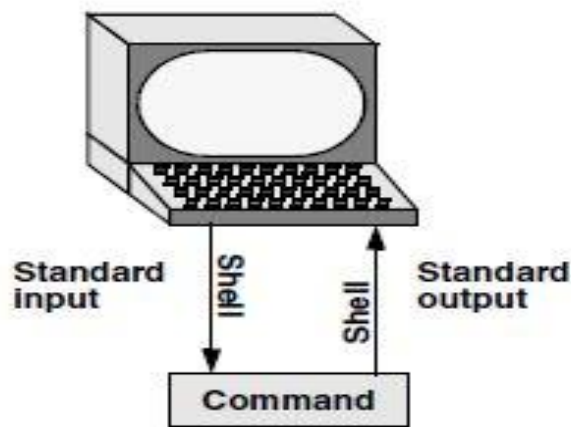
If it finds an executable file with the same name as the command, the shell starts a new process. A process is the execution of a command by Linux. The shell makes each command-line argument, including options and the name of the command, available to the called program. While the command is executing, the shell waits for the process to finish. At this point the shell is in an inactive state called sleep. When the program finishes execution, it passes its exit status to the shell. The shell then returns to an active state (wakes up), issues a prompt, and waits for another command.

Editing the Command Line

You can repeat and edit previous commands and edit the current command line.

7.2 Standard Input and Standard Output

Standard output is a place a program can send information, such as text. The program never “knows” where the information it sends to standard output is going. The information can go to a printer, an ordinary file, or the screen. By default, the shell directs standard output from a command to the screen.



Standard input is a place that a program gets information from. As with standard output the program never “knows” where the information comes from. In addition to standard input and standard output, a running program normally has a place to send error messages: standard error.

The Screen as a File

Linux have an additional type of file: a device file. A device file resides in the Linux file structure, usually in the /dev directory, and represents a peripheral device, such as a screen and keyboard, printer, or disk drive. The device name that the who utility displays after your username is the filename of the screen and keyboard.

The Keyboard and Screen as Standard Input and Standard Output

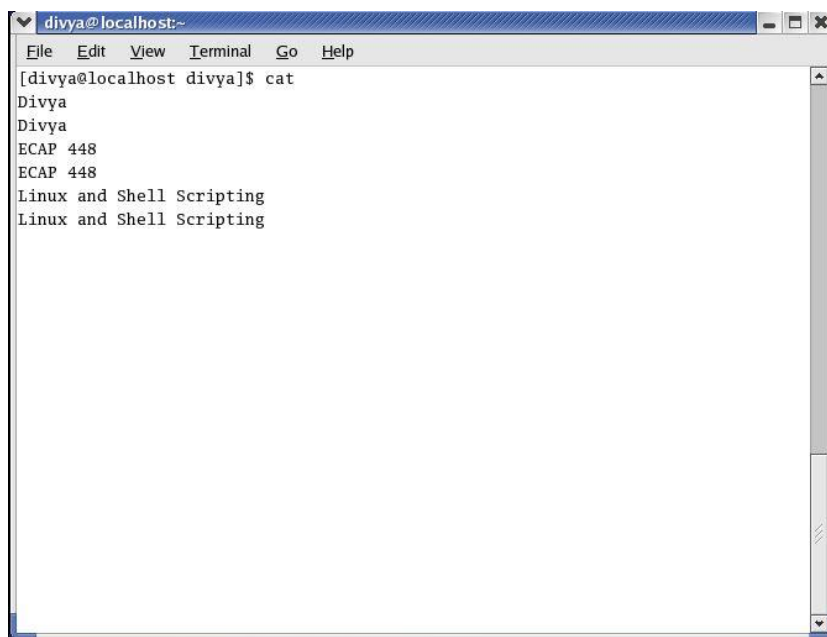
When you first log in, the shell directs standard output of your commands to the device file that represents the screen. Directing output in this manner causes it to appear on the screen.

cat utility

The cat utility provides a good example of the way the keyboard and screen function as standard input and standard output, respectively. When you use cat, it copies a file to standard output. Because the shell directs standard output to the screen, cat displays the file on the screen. Up to this point cat has taken its input from the filename (argument) you specify on the command line.

Unit 07: The Shell and Popular Editors

When you do not give `cat` any argument (that is, when you give the command `cat` followed immediately by RETURN), `cat` takes its input from standard input. Thus, when called without an argument, `cat` copies standard input to standard output, one line at a time.



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ cat  
Divya  
Divya  
ECAP 448  
ECAP 448  
Linux and Shell Scripting  
Linux and Shell Scripting
```

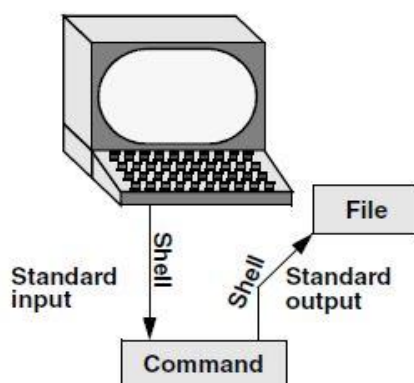
Because the shell associates `cat`'s standard input with the keyboard and `cat`'s standard output with the screen, when you type a line of text `cat` copies the text from standard input (the keyboard) to standard output (the screen). The `cat` utility keeps copying text until you enter CONTROL-D on a line by itself. Pressing CONTROL-D sends an EOF (end of file) signal to `cat` to indicate that it has reached the end of standard input and there is no more text for it to copy. The `cat` utility then finishes execution and returns control to the shell, which displays a prompt.

7.3 Redirection

The term redirection encompasses the various ways you can cause the shell to alter where standard input of a command comes from and where standard output goes to. By default, the shell associates standard input and standard output of a command with the keyboard and the screen. You can cause the shell to redirect standard input or standard output of any command by associating the input or output with a command or file other than the device file representing the keyboard and the screen.

Redirecting Standard Output

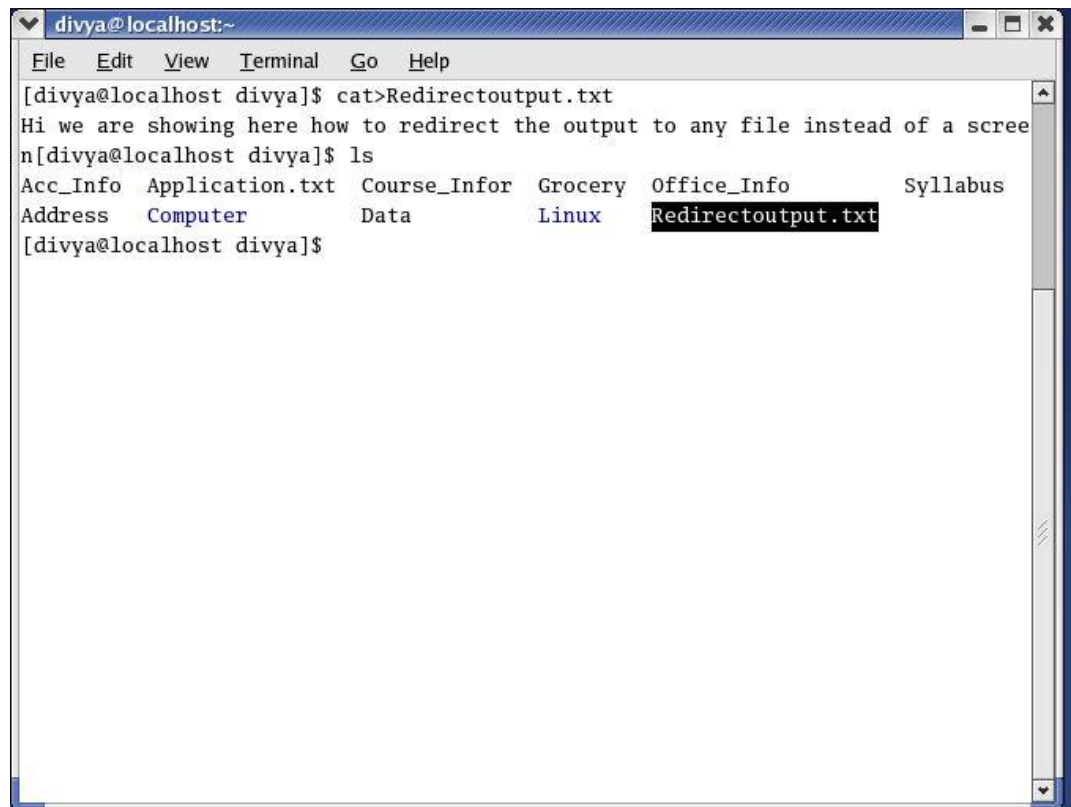
The redirect output symbol (`>`) instructs the shell to redirect the output of a command to the specified file instead of to the screen.



Linux and Shell Scripting

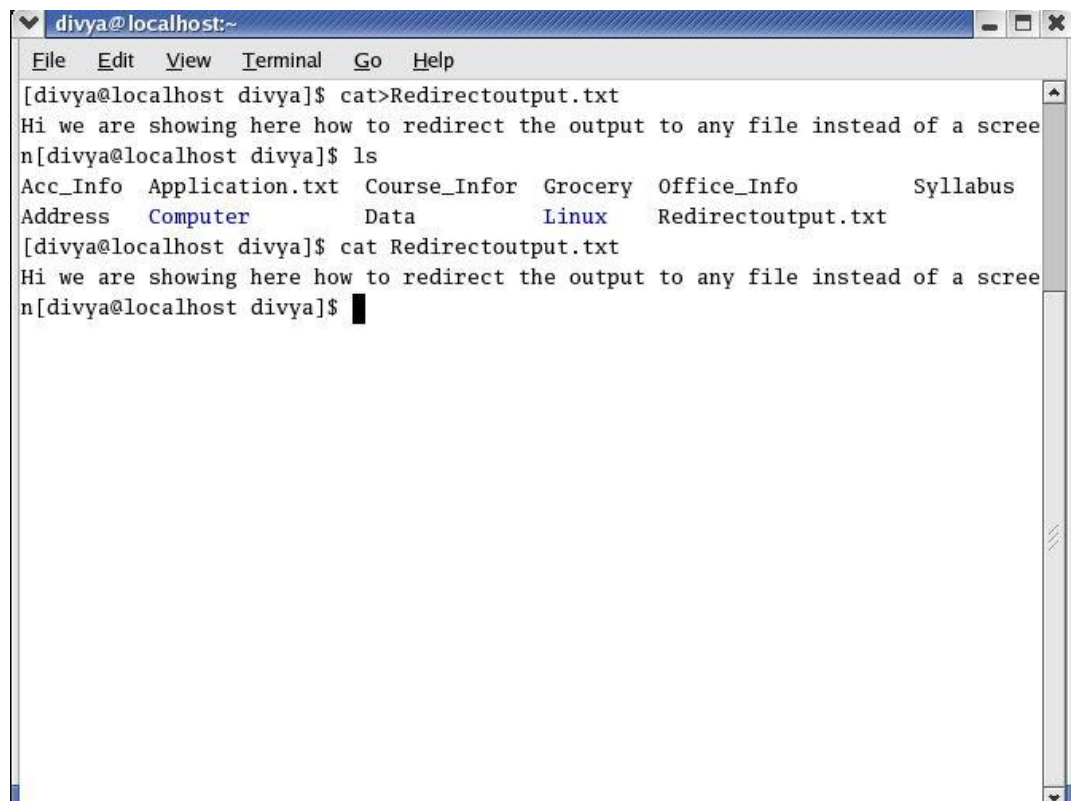
The format of a command line that redirects output is `command [arguments] > filename`

where `command` is any executable program (such as an application program or a utility), `arguments` are optional arguments, and `filename` is the name of the ordinary file the shell redirects the output to.



A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The user enters the command `cat>Redirectoutput.txt`, and the output 'Hi we are showing here how to redirect the output to any file instead of a screen' is written to the file. Then, the user enters `ls`, and the output lists files including 'Redirectoutput.txt', which is highlighted with a black background.

```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ cat>Redirectoutput.txt  
Hi we are showing here how to redirect the output to any file instead of a screen  
[divya@localhost divya]$ ls  
Acc_Info  Application.txt  Course_Infor  Grocery  Office_Info  Syllabus  
Address   Computer         Data          Linux    Redirectoutput.txt  
[divya@localhost divya]$
```



The same terminal window shows the user entering `cat Redirectoutput.txt`. The output is the same text as before, but it is now read from the file 'Redirectoutput.txt' instead of being redirected to it.

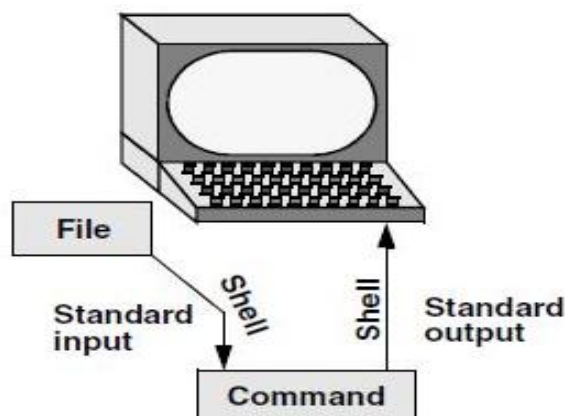
```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ cat>Redirectoutput.txt  
Hi we are showing here how to redirect the output to any file instead of a screen  
[divya@localhost divya]$ ls  
Acc_Info  Application.txt  Course_Infor  Grocery  Office_Info  Syllabus  
Address   Computer         Data          Linux    Redirectoutput.txt  
[divya@localhost divya]$ cat Redirectoutput.txt  
Hi we are showing here how to redirect the output to any file instead of a screen  
[divya@localhost divya]$
```


The redirect output symbol on the command line causes the shell to associate cat's standard output with the redirectoutput.txt file specified on the command line. Redirecting standard output from cat is a handy way to create a file without using an editor. The drawback is that once you enter a line and press RETURN, you cannot edit the text. While you are entering a line, the erase and kill keys work to delete text. This procedure is useful for creating short, simple files. The cat is used and the redirect output symbol to catenate (join one after the other – the derivation of the name of the cat utility) several files into one larger file.

```
divya@localhost:~  
File Edit View Terminal Go Help  
Punjab  
India  
144411  
[divya@localhost divya]$ cat Course_Infor  
Course 1: Computer Organization and Architecture  
Course 2: The Linux and Shell Scripting  
  
Course 1 Completed  
Course 2 In process  
[divya@localhost divya]$ cat Office_Info Course_Infor > Complete.txt  
[divya@localhost divya]$ cat Complete.txt  
Cabin No 11  
Room No 210  
Block No 21  
Lovely Professional University  
Chaheru  
Phagwara  
Punjab  
India  
144411  
Course 1: Computer Organization and Architecture  
Course 2: The Linux and Shell Scripting  
  
Course 1 Completed  
Course 2 In process  
[divya@localhost divya]$
```

Redirecting Standard Input

The redirect input symbol (<) instructs the shell to redirect a command's input to come from the specified file instead of from the keyboard.



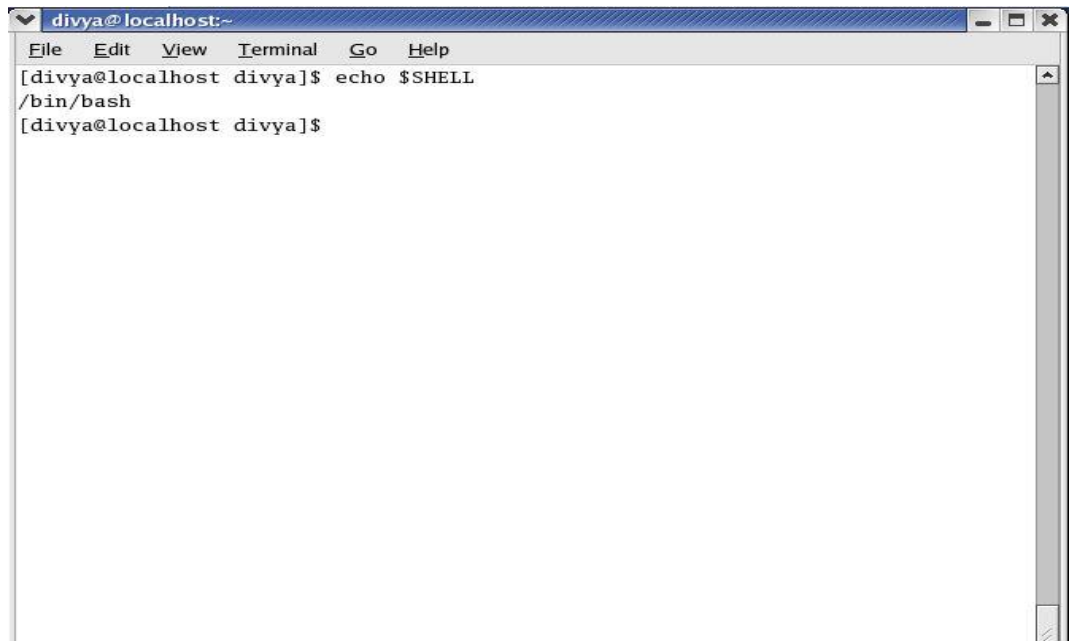
The format of a command line that redirects input is: command [arguments] < filename

Linux and Shell Scripting

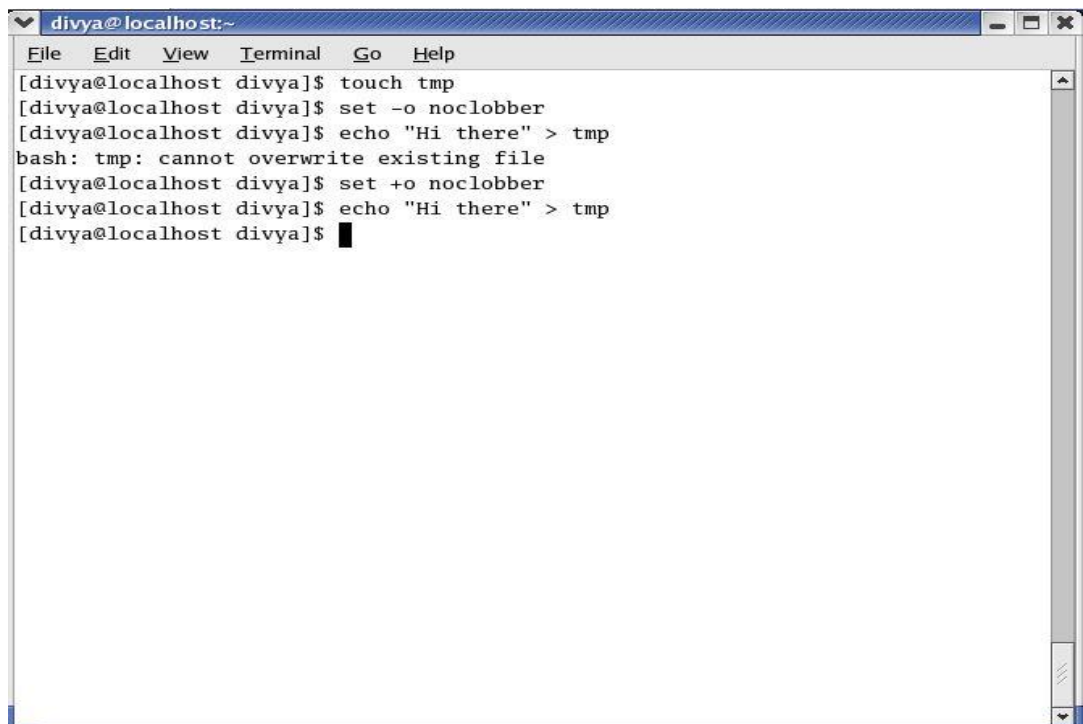
where command is any executable program (such as an application program or a utility), arguments are optional arguments, and filename is the name of the ordinary file the shell redirects the input from.

noclobber: Avoids Overwriting Files

The shell provides the noclobber feature that prevents overwriting a file using redirection. Under bash you can enable this feature by setting noclobber using the command `set -o noclobber`. The same command with `+o` unsets noclobber. Under tcsh use `set noclobber` and `unset noclobber`. With noclobber set, if you redirect output to an existing file, the shell displays an error message and does not execute the command.

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The terminal shows the following commands and output:

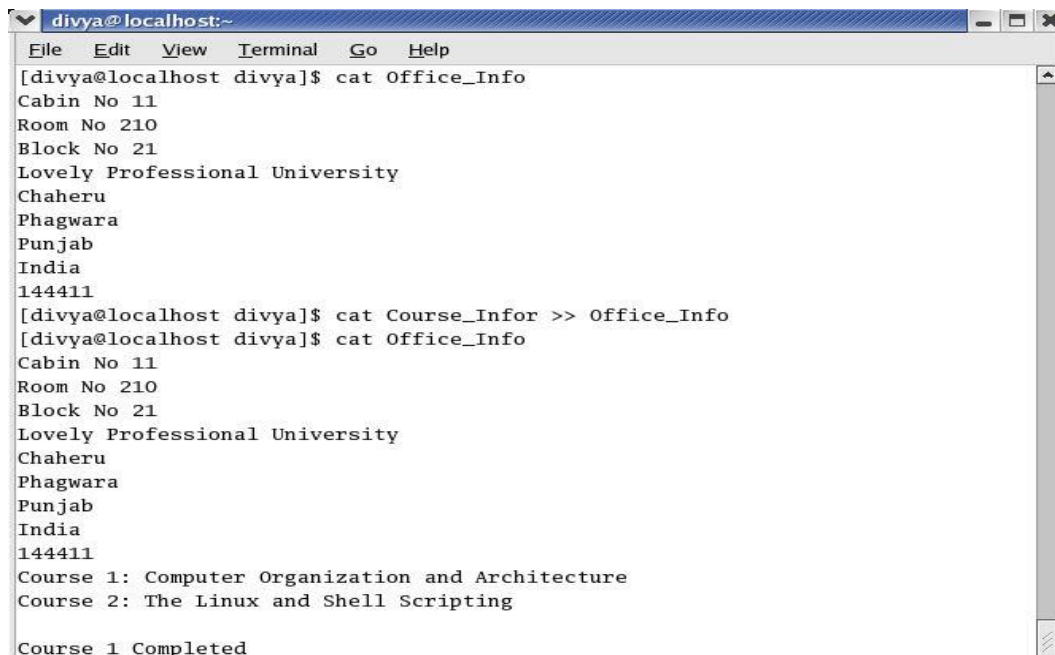
```
[divya@localhost divya]$ echo $SHELL
/bin/bash
[divya@localhost divya]$
```

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The terminal shows the following commands and output:

```
[divya@localhost divya]$ touch tmp
[divya@localhost divya]$ set -o noclobber
[divya@localhost divya]$ echo "Hi there" > tmp
bash: tmp: cannot overwrite existing file
[divya@localhost divya]$ set +o noclobber
[divya@localhost divya]$ echo "Hi there" > tmp
[divya@localhost divya]$
```

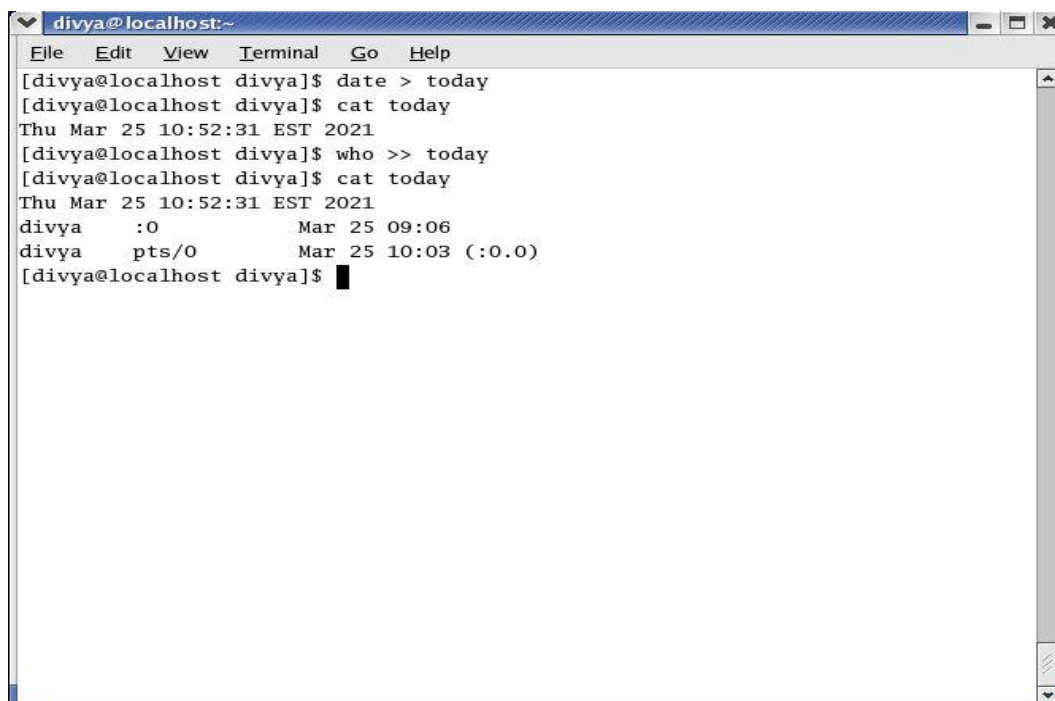
Appending Standard Output to a File

The append output symbol (>>) causes the shell to add new information to the end of a file, leaving existing information intact. This symbol provides a convenient way of catenating two files into one.

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The terminal shows the following commands and output:

```
[divya@localhost divya]$ cat Office_Info
Cabin No 11
Room No 210
Block No 21
Lovely Professional University
Chaheru
Phagwara
Punjab
India
144411
[divya@localhost divya]$ cat Course_Infor >> Office_Info
[divya@localhost divya]$ cat Office_Info
Cabin No 11
Room No 210
Block No 21
Lovely Professional University
Chaheru
Phagwara
Punjab
India
144411
Course 1: Computer Organization and Architecture
Course 2: The Linux and Shell Scripting

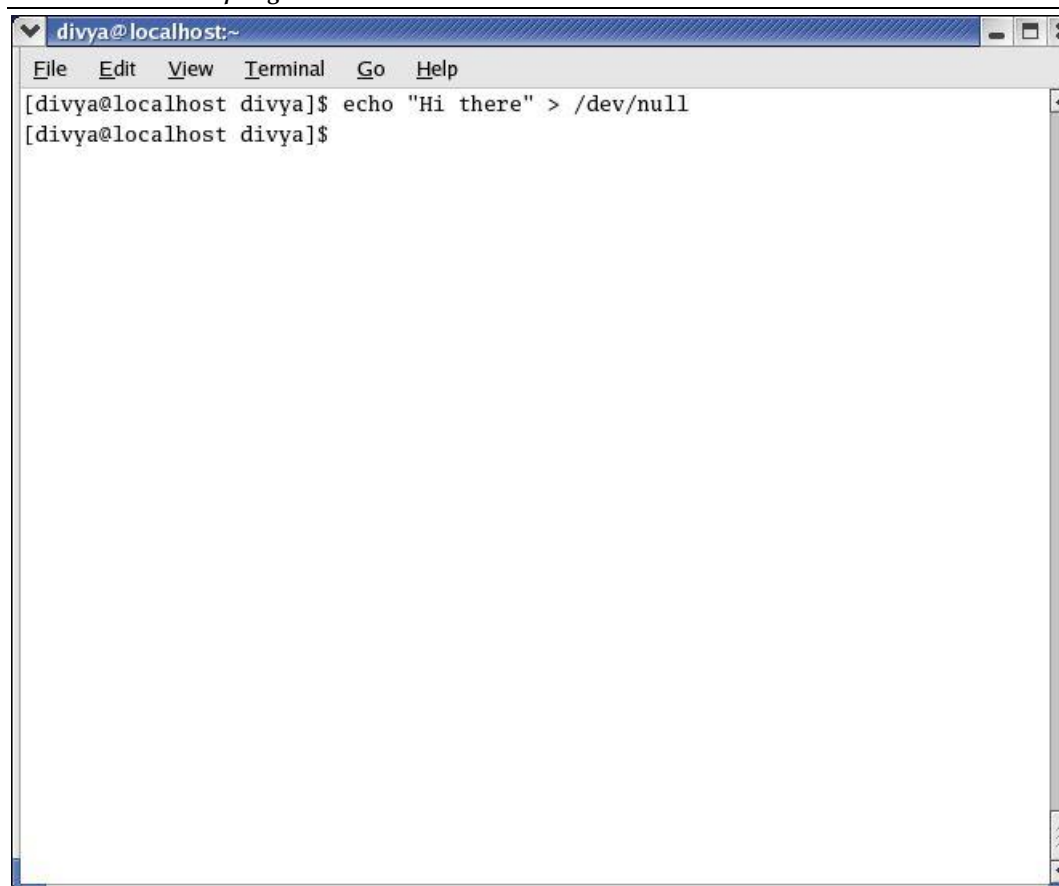
Course 1 Completed
```

A terminal window titled 'divya@localhost:~' with a menu bar (File, Edit, View, Terminal, Go, Help). The terminal shows the following commands and output:

```
[divya@localhost divya]$ date > today
[divya@localhost divya]$ cat today
Thu Mar 25 10:52:31 EST 2021
[divya@localhost divya]$ who >> today
[divya@localhost divya]$ cat today
Thu Mar 25 10:52:31 EST 2021
divya      :0                Mar 25 09:06
divya      pts/0            Mar 25 10:03 (:0.0)
[divya@localhost divya]$
```

/dev/null: Making Data Disappear

The /dev/null device is a data sink, commonly referred to as a bit bucket. You can redirect output that you do not want to keep or see to /dev/null and the output will disappear without a trace.



```
divya@localhost:~  
File Edit View Terminal Go Help  
[divya@localhost divya]$ echo "Hi there" > /dev/null  
[divya@localhost divya]$
```

7.4 Pipes

The shell uses a pipe to connect standard output of one command to standard input of another command. A pipe (sometimes referred to as a pipeline) has the same effect as redirecting standard output of one command to a file and then using that file as standard input to another command. The symbol for a pipe is a vertical bar (|). The syntax of a command line using a pipe is

```
command_a [arguments] | command_b [arguments]
```

The preceding command line uses a pipe on a single command line to generate the same result as the following three command lines:

```
command_a [arguments] > temp  
command_b [arguments] < temp  
rm temp
```

7.5 Filters

A filter is a command that processes an input stream of data to produce an output stream of data. A command line that includes a filter uses a pipe to connect standard output of one command to the filter's standard input. Another pipe connects the filter's standard output to standard input of another command. Not all utilities can be used as filters. In this example, `sort` is a filter, taking standard input from standard output of `who` and using a pipe to redirect standard output to standard input of `lpr`. This command line sends the sorted output of `who` to the printer: `$ who | sort | lpr`

tee: Sends Output in Two Directions

The `tee` utility copies its standard input both to a file and to standard output. This utility is aptly named: It takes a single stream of input and sends the output in two directions.

```
$ who | tee who.out | grep sam
sam      console   Mar 24 05:00
$ cat who.out
sam      console   Mar 24 05:00
max      pts/4       Mar 24 12:23
max      pts/5       Mar 24 12:33
zach     pts/7       Mar 23 08:45
```

The output of `who` is sent via a pipe to standard input of `tee`. The `tee` utility saves a copy of standard input in a file named `who.out` and sends a copy to standard output. Standard output of `tee` goes via a pipe to standard input of `grep`, which displays only those lines containing the string `sam`.

7.6 Running a Command in the Background

All commands up to this point have been run in the foreground. When you run a command in the foreground, the shell waits for it to finish before displaying another prompt and allowing you to continue. When you run a command in the background, you do not have to wait for the command to finish before running another command. A job is a series of one or more commands that can be connected by pipes. You can have only one foreground job in a window or on a screen, but you can have many background jobs. Multitasking is the running of more than one job at a time.

To run a job in the background, type an ampersand (&) just before the RETURN that ends the command line. The shell assigns a small number to the job and displays this job number between brackets. Following the job number, the shell displays the process identification (PID) number—a larger number assigned by the operating system. Each of these numbers identifies the job running in the background. The shell then displays another prompt, and you can enter another command. When the background job finishes, the shell displays a message giving both the job number and the command line used to run the command. This command runs in the background; it sends the output of `ls` through a pipe to `lpr`, which sends it to the printer.

```
$ ls -l | lpr&
[1] 22092
$
```

The `[1]` following the command line indicates that the shell has assigned job number 1 to this job. The `22092` is the PID number of the first command in the job. When this background job completes execution, the shell displays the message `[1]+ Done ls -l | lp`

Moving a Job from the Foreground to the Background

You can suspend a foreground job (stop it from running without aborting the job) by pressing the suspend key, usually `CONTROL-Z`. The shell then stops the process and disconnects standard input from the keyboard. You can put a suspended job in the background and restart it by using the `bg` command followed by the job number. You do not need to specify the job number when there is only one stopped job. Only the foreground job can take input from the keyboard.

To connect the keyboard to a program running in the background, you must bring it to the foreground. To do so, type `fg` without any arguments when only one job is in the background. When more than one job is in the background, type `fg`, or a percent sign (%), followed by the number of the job you want to bring into the foreground. The shell displays the command you used to start the job (prompted in the following example), and you can enter any input the program requires to continue:

```
bash $ fg 1
promptme
```

kill: Aborting a Background Job

The interrupt key (usually CONTROL-C) cannot abort a background process; you must use kill for this purpose. Follow kill on the command line with either the PID number of the process you want to abort or a percent sign (%) followed by the job number. If you forget a PID number, you can use the ps (process status) utility to display it. The following example runs a tail -f outfile command as a background job, uses ps to display the PID number of the process, and aborts the job with kill:

```
$ tail -f outfile&
[1] 18228
$ ps | grep tail
18228 pts/4 00:00:00 tail
$ kill 18228
[1]+ Terminated tail -f outfile
$
```

If you forget a job number, you can use the jobs command to display a list of job numbers. The next example is like the previous one except it uses the job number instead of the PID number to identify the job to be killed. Sometimes the message saying the job is terminated does not appear until you press RETURN after the RETURN that executes the kill command.

```
$ tail -f outfile&
[1] 18236
$ bigjob&
[2] 18237
$ jobs
[1]- Running tail -f outfile&
[2]+ Running bigjob&
$ kill %1
$ RETURN
[1]- Terminated tail -f outfile
$
```

7.7 Filename Generation/Pathname Expansion

When you give the shell abbreviated filenames that contain special characters, also called metacharacters, the shell can generate filenames that match the names of existing files. These special characters are also referred to as wildcards because they act much as the jokers do in a deck of cards. When one of these characters appears in an argument on the command line, the shell expands that argument in sorted order into a list of filenames and passes the list to the program called by the command line. Filenames that contain these special characters are called ambiguous file references because they do not refer to any one specific file. The process that the shell performs on these filenames is called pathname expansion or globbing.

Ambiguous file references refer to a group of files with similar names quickly, saving the effort of typing the names individually. They can also help find a file whose name you do not remember in its entirety. If no filename matches the ambiguous file reference, the shell generally passes the unexpanded reference—special characters and all—to the program.

The ? Special Character

The question mark (?) is a special character that causes the shell to generate filenames. It matches any single character in the name of an existing file. The following command uses this special character in an argument to the lpr utility: \$ lprmemo?.

Unit 07: The Shell and Popular Editors

The shell expands the `memo?` argument and generates a list of files in the working directory that have names composed of `memo` followed by any single character. The shell then passes this list to `lpr`. The `lpr` utility never “knows” the shell generated the filenames it was called with. If no filename matches the ambiguous file reference, the shell passes the string itself (`memo?`) to `lpr` or, if it is set up to do so, passes a null string. The following example uses `ls` first to display the names of all files in the working directory and then to display the filenames that `memo?` matches:

```
$ ls
mem  memo12  memo9  memomax  newmemo5
memo  memo5  memoa  memos

$ ls memo?
memo5  memo9  memoa  memos
```

The `memo?` ambiguous file reference does not match `mem`, `memo`, `memo12`, `memomax`, or `newmemo5`. You can also use a question mark in the middle of an ambiguous file reference:

```
$ ls
7may4report  may4report  mayqreportmay_report
may14report  may4report.79  mayreportmay.report

$ ls may?report
may.report  may4report  may_reportmayqreport
```

The * Special Character

The asterisk (*) performs a function like that of the question mark but matches any number of characters, including zero characters, in a filename.

```
$ ls
amemo  memo.0612  memosally  memsam  user.memo  mem  memoa  memosam.0620
sallymemo  memo  memorandum  memosam.keep  typescript

$ echo memo*
memo  memo.0612  memoa  memorandum  memosally  memosam.0620  memosam.keep

$ echo *mo
amemo  memo  sallymemouser.memo

$ echo *sam*
memosam.0620  memosam.keepmemsam
```

The ambiguous file reference `memo*` does not match `amemo`, `mem`, `sallymemo`, or `user.memo`. Like the question mark, an asterisk does not match a leading period in a filename. The `-a` option causes `ls` to display hidden filenames. The command `echo *` does not display `.` (the working directory), `..` (the parent of the working directory), `.aaa`, or `.profile`. In contrast, the command `echo .*` displays only those four names:

```
$ ls
aaamemo.sally  sally.0612  thurs
memo.0612  report  saturday
```



```
$ ls -a
.      .aaaaaamemo.sally  sally.0612  thurs
..     .profile    memo.0612    report    saturday

$ echo *
aaa    memo.0612    memo.sally  report sally.0612 saturdaythurs

$ echo .*
.      ..     .aaa     .profile
```

In the following example, `.p*` does not match `memo.0612`, `private`, `reminder`, or `report`. The `ls .*` command causes `ls` to list `.private` and `.profile` in addition to the contents of the `.` directory (the working directory) and the `..` directory (the parent of the working directory). When called with the same argument, `echo` displays the names of files (including directories) in the working directory that begin with a dot (`.`), but not the contents of directories.

```
$ ls -a
.      .private  memo.0612  reminder
..     .profile  private    report

$ echo .p*
.private    .profile

$ ls .*
.private    .profile
.:
memo.0612   private    reminder    report
...:
.
.

$ echo .*
.      ..     .private    .profile
```

You can plan to take advantage of ambiguous file references when you establish conventions for naming files. For example, when you end all text filenames with `.txt`, you can reference that group of files with `*.txt`. The next command uses this convention to send all text files in the working directory to the printer. The ampersand (`&`) causes `lpr` to run in the background.

```
$ lpr *.txt &
```

The [] Special Characters

A pair of brackets surrounding a list of characters causes the shell to match filenames containing the individual characters. Whereas `memo?` matches `memo` followed by any character, `memo[17a]` is more restrictive: It matches only `memo1`, `memo7`, and `memoa`. The brackets define a character class that includes all the characters within the brackets. The shell expands an argument that includes a

Unit 07: The Shell and Popular Editors

character-class definition, by substituting each member of the character class, one at a time, in place of the brackets and their contents. The shell then passes the list of matching filenames to the program it is calling. Each character-class definition can replace only a single character within a filename. The brackets and their contents are like a question mark that substitutes only the members of the character class. The first of the following commands lists the names of all files in the working directory that begin with a, e, i, o, or u. The second command displays the contents of the files named page2.txt, page4.txt, page6.txt, and page8.txt.

```
$ echo [aeiou]*
...
$ less page[2468].txt
...
```

A hyphen within brackets defines a range of characters within a character-class definition. For example, [6-9] represents [6789], [a-z] represents all lowercase letters in English, and [a-zA-Z] represents all letters, both uppercase and lowercase, in English. The following command lines show three ways to print the files named part0, part1, part2, part3, and part5. Each of these command lines causes the shell to call lpr with five filenames:

```
$ lpr part0 part1 part2 part3 part5
$ lpr part[01235]
$ lpr part[0-35]
```

The following command line prints 39 files, part0 through part38:

```
$ lprpart[0-9] part[12][0-9] part3[0-8]
```

The first of the following commands lists the files in the working directory whose names start with a through m.

```
$ echo [a-m]*
...
```

The second lists files whose names end with x, y, or z.

```
$ echo *[x-z]
...
```

The ls utility cannot interpret ambiguous file references. First ls is called with an argument of ?old. The shell expands ?old into a matching filename, hold, and passes that name to ls.

```
$ ls ?old
hold
```

The second command is the same as the first, except the ? is quoted.

```
$ ls \?old
```

```
ls: ?old: No such file or directory
```

The shell does not recognize this question mark as a special character and passes it to ls. The ls utility generates an error message saying that it cannot find a file named ?old (because there is no file named ?old).

7.8 Builtins

A builtin is a utility (also called a command) that is built into a shell. Each of the shells has its own set of builtins. When it runs a builtin, the shell does not fork a new process. Consequently, builtins run more quickly and can affect the environment of the current shell. Because builtins are used in the same way as utilities, you will not typically be aware of whether a utility is built into the shell or is a standalone utility. The `echo` utility, for example, is a shell builtin. The shell always executes a shell builtin before trying to find a command or utility with the same name.

Listing bash builtins

To display a list of bash builtins, give the command `info bash` and select the Shell Builtin Commands menu. Then select the Bourne Shell Builtins and/or Bash Builtins menus. The bash info page is part of the bash-doc package—you can view only the man page (even using `info`) if this package is not installed. Because bash was written by GNU, the info page has better information than does the man page.

Listing tcshbuiltins

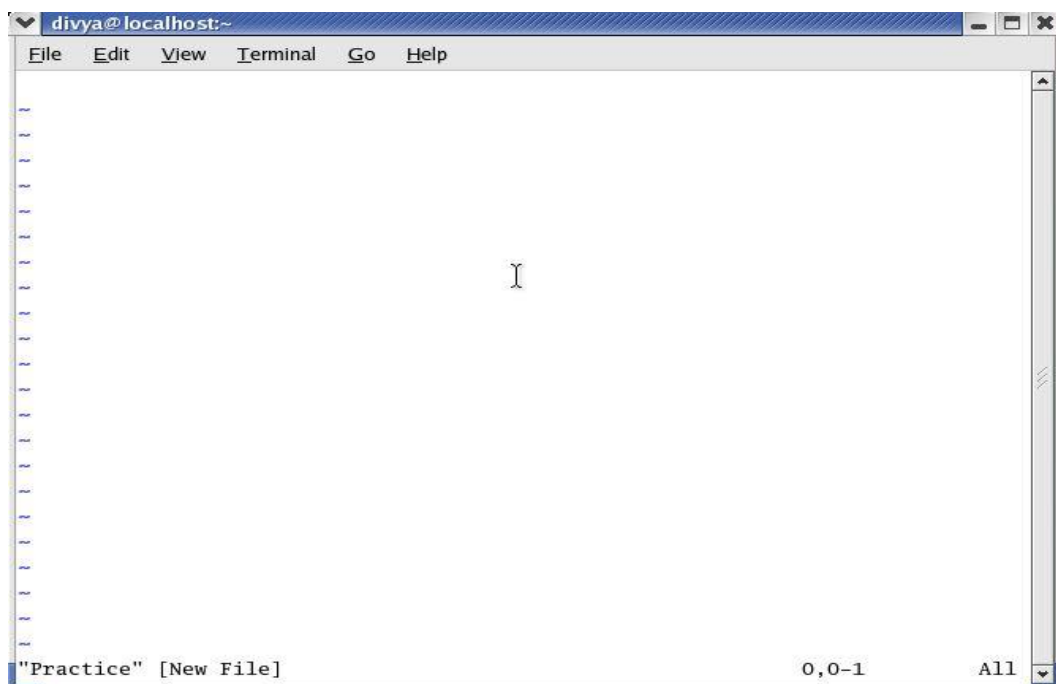
For tcsh, give the command `man tcsh` to display the tcsh man page and then search with the following command: `/Builtin commands$` (search for the string at the end of a line).

7.9 vim

The vim editor is not a text formatting program. It does not justify margins or provide the output formatting features of a sophisticated word processing system such as OpenOffice.org Writer. Vim is a sophisticated text editor meant to be used to write code (C, HTML, Java, etc.), short notes, and input to a text formatting system, such as `groff` or `troff`.

Starting vim

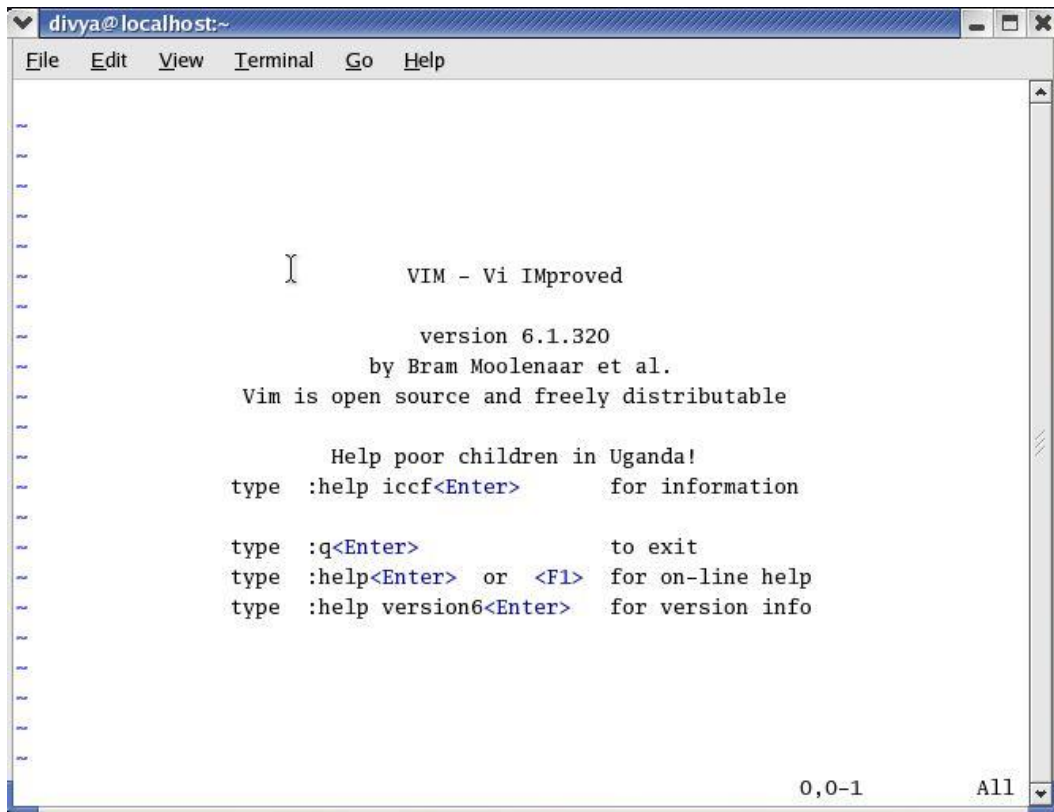
Start vim with the following command to create and edit a file named `practice`: `$ vim practice`. The tildes in the starting indicates that the page is blank.



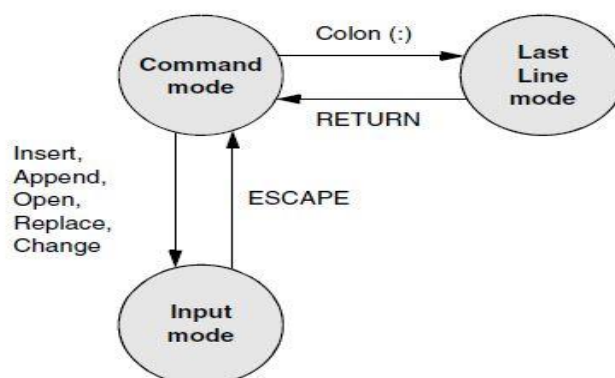
Unit 07: The Shell and Popular Editors

If you call vim without specifying a filename on the command line, vim assumes that you are a novice and tells you how to get started.

```
$ vim
```



7.10 Modes in vim



Two of vim's modes of operation are Command mode (also called Normal mode) and Input mode. While vim is in Command mode, you can give vim commands. For example, you can delete text or exit from vim. You can also command vim to enter Input mode. In Input mode, vim accepts anything you enter as text and displays it on the screen. The colon (:) in the preceding command puts vim into another mode, Last Line mode. While in this mode, vim keeps the cursor on the bottom line of the screen. When you press RETURN to finish entering the command, vim restores the cursor to its place in the text. Press ESCAPE to return vim to Command mode. By default, the vim editor informs you about which mode it is in: It displays INSERT at the lower-left corner of the screen while it is in Insert mode.

Entering Text

When you start vim, you must put it in Input mode before you can enter text. To put vim in Input mode, press the i (insert before cursor) key or the a (append after cursor) key. If you are not sure whether vim is in Input mode, press the ESCAPE key; vim returns to Command mode if it is in Input mode or beeps, flashes, or does nothing if it is already in Command mode. You can put vim back in Input mode by pressing the i or a key again. While vim is in Input mode, you can enter text by typing on the keyboard. If the text does not appear on the screen as you type, vim is not in Input mode.

Correcting Text

The keys that back up and correct a shell command line serve the same functions when vim is in Input mode.

Key	Results
CONTROL-H	Erase
CONTROL-U	Line kill
CONTROL-W	Word kill
Dd, dw, x	Remove the incorrect text
I, a, o, O	Insert the correct text

To change the word pressing to hitting in, you might use the ARROW keys to move the cursor until it is on top of the p in pressing. Then give the command dw to delete the word pressing. Put vim in Input mode by giving an i command, enter the word hitting followed by a SPACE, and press ESCAPE. The word is changed, and vim is in Command mode, waiting for another command. A shorthand for the two commands dw followed by the i command is cw (Change word). The cw command puts vim into Input mode.

Ending the Editing Session

While you are editing, vim keeps the edited text in an area named the Work buffer. Use the ZZ command (you must use uppercase Zs) to write the newly entered text to the disk. You can exit with :q! if you do not want to save your work.

Moving the Cursor

To delete, insert, and correct text, you need to move the cursor on the screen. While vim is in Command mode, you can use the RETURN key, the SPACE bar, and the ARROW keys to move the cursor. You can use the h, j, k, and l (lowercase "l") keys to move the cursor left, down, up, and right, respectively.

Deleting Text

If you want to delete a single character, then press x. If a word needs to be deleted, press dw and for the deletion of line of text, press dd.

Undoing Mistakes

Give the command u (Undo) immediately after the command you want to undo. If you give the u command again, vim will undo the command you gave before the one it just undid. You can use this technique to back up over many of your actions. If you undo a command, you did not mean to undo, give a Redo command: CONTROL-R or :redo (followed by a RETURN). As with the Undo command, you can give the Redo command many times in a row.

Entering Additional Text

When you want to insert new text within existing text, move the cursor so it is on the character that follows the new text you plan to enter. Then give the `i` (Insert) command to put vim in Input mode, enter the new text, and press `ESCAPE` to return vim to Command mode. Alternatively, you can position the cursor on the character that precedes the new text and use the `a` (Append) command. To enter one or more lines, position the cursor on the line above where you want the new text to go. Give the command `o` (Open). The vim editor opens a blank line below the line the cursor was on, puts the cursor on the new, empty line, and enters Input mode. Enter the new text, ending each line with a `RETURN`. When you are finished entering text, press `ESCAPE` to return vim to Command mode. The `O` command works in the same way as the `o` command, except it opens a blank line above the current line.

7.11 Introduction to vim Features

There are various useful features of vim which are of great help:

- Online help,
- Modes of operation,
- The work buffer,
- Emergency procedures,
- Other vim features.

Online Help

Give the command: `help` feature to display information about feature. As you scroll through the various help texts, you will see words with a bar on either side, such as `|tutor|`. These words are active links: Move the cursor on top of an active link and press `CONTROL-]` to jump to the linked text. Use `CONTROL-o` (lowercase “o”) to jump back to where you were in the help text. You can also use the active link words in place of feature. For example, you might see the reference `|credits|`; you could enter `:help credits RETURN` to read more about credits. Enter `:q!` to close a help window. You can also give the command `:help doc-file-list` to view a complete list of the help files.

Modes of Operation

The current character is the character the cursor is on. The current line is the line the cursor is on. The status line is the last or bottom line of the screen. This line is reserved for Last Line mode and status information. Text you are editing does not appear on this line. The vim editor is part of the ex-editor, which has five modes of operation:

- ex Command mode
- ex Input mode
- vim Command mode
- vim Input mode
- vim Last Line mode

While in Command mode, vim accepts keystrokes as commands, responding to each command as you enter it. It does not display the characters you type in this mode. While in Input mode, vim accepts and displays keystrokes as text that it eventually puts into the file you are editing. All commands that start with a colon (`:`) put vim in Last Line mode. The colon moves the cursor to the status line of the screen, where you enter the rest of the command. When you give a command in Command mode, you do not terminate the command with a `RETURN`. In contrast, you must terminate all Last Line mode commands with a `RETURN`. When an editing session begins, vim is in Command mode. Several commands, including Insert and Append, put vim in Input mode. When you press the `ESCAPE` key, vim always reverts to Command mode. The Change and Replace commands combine the Command and Input modes. The Change command deletes the text you

Linux and Shell Scripting

want to change and puts vim in Input mode so you can insert new text. The Replace command deletes the character(s) you overwrite and inserts the new one(s) you enter.

The vim editor displays status information on the bottom line of the display area. This information includes error messages, information about the deletion or addition of blocks of text, and file status information. In addition, vim displays Last Line mode commands on the status line. Sometimes the screen may become garbled or overwritten. When vim puts characters on the screen, it sometimes leaves @ on a line instead of deleting the line. When output from a program becomes intermixed with the display of the Work buffer, things can get even more confusing. The output does not become part of the Work buffer but affects only the display. If the screen gets overwritten, press ESCAPE to make sure vim is in Command mode, and press CONTROL-L to redraw (refresh) the screen. If the end of the file is displayed on the screen, vim marks lines that would appear past the end of the file with a tilde (~) at the left of the screen. While vim is in Input mode, you can use the erase and line kill keys to back up over text so you can correct it. You can also use CONTROL-W to back up over words.

Work Buffer

The vim editor does all its work in the Work buffer. At the beginning of an editing session, vim reads the file you are editing from the disk into the Work buffer. During the editing session, it makes all changes to this copy of the file but does not change the file on the disk until you write the contents of the Work buffer back to the disk. Normally when you end an editing session, you tell vim to write the contents of the Work buffer, which makes the changes to the text final. If you accidentally end an editing session without writing out the contents of the Work buffer, your work is lost. However, if you unintentionally make some major changes (such as deleting the entire contents of the Work buffer), you can end the editing session without implementing the changes. To look at a file but not to change it while you are working with vim, you can use the view utility: \$ view filename. Calling the view utility is the same as calling the vim editor with the -R (readonly) option.

The vim editor operates on files of any format. The total length of the file is limited only by available disk space and memory. The vim editor allows you to open, close, and hide multiple windows, each of which allows you to edit a different file. Give the command :help windows to display a complete list of windows commands.

Abnormal Termination of an Editing Session and recovering after a crash (Emergency Procedures)

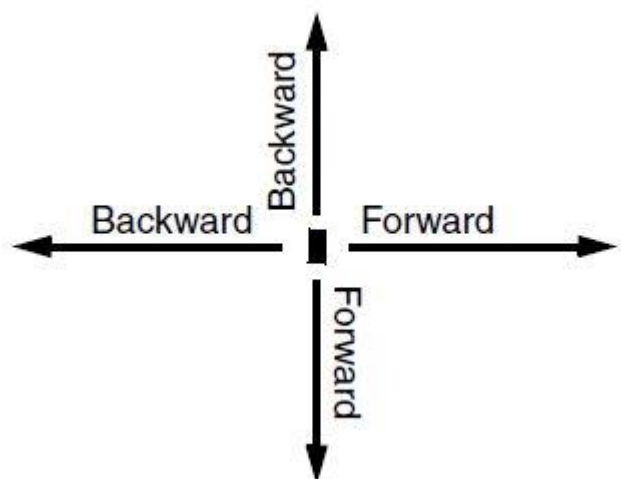
You can end an editing session in one of two ways: When you exit from vim, you can save the changes you made during the editing session or you can abandon those changes. You can use the ZZ or :wq command from Command mode to save the changes and exit from vim. To end an editing session without writing out the contents of the Work buffer, give the following command:

:q! Use the :q! command cautiously. When you use this command to end an editing session, vim does not preserve the contents of the Work buffer, so you will lose any work you did since the last time you wrote the Work buffer to disk.:w filename: Use this to save the file with a name.

The vim editor temporarily stores the file you are working on in a swap file. If the system crashes while you are editing a file with vim, you can often recover its text from the swap file. If someone else is editing the file, quit or open the file as a read only file. With the -r option, vim displays a list of swap files it has saved (some may be old). If your work was saved, give the same command followed by a SPACE and the name of the file. Give the command :w filename immediately to save the salvaged copy of the Work buffer to disk under a name different from the original file; then check the recovered file to make sure it is OK.

7.12 Command Mode

While vim is in Command mode, you can position the cursor over any character on the screen.

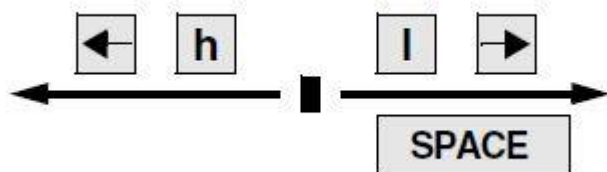


Forward means toward the right and bottom of the screen and the end of the file. Backward means toward the left and top of the screen and the beginning of the file.

Moving the Cursor

When you use a command that moves the cursor forward past the end (right) of a line, the cursor generally moves to the beginning (left) of the next line. When you move it backward past the beginning of a line, the cursor generally moves to the end of the previous line. Sometimes a line in the Work buffer may be too long to appear as a single line on the screen. In such a case vim wraps the current line onto the next line. You can move the cursor through the text by any Unit of Measure (that is, character, word, line, sentence, paragraph, or screen). If you precede a cursor-movement command with a number, called a Repeat Factor, the cursor moves that number of units through the text.

Moving the Cursor by Characters



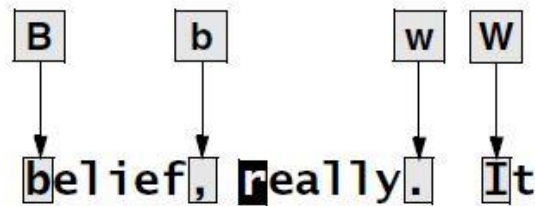
Key	Results
Space bar, l and Right Arrow Key	Forward
h and Left Arrow Key	Backward

For example, the command 7 SPACE or 7l moves the cursor seven characters to the right.

Moving the Cursor to a Specific Character

You can move the cursor to the next occurrence of a specified character on the current line by using the Find command. For example, the following command moves the cursor from its current position to the next occurrence of the character a, if one appears on the same line: fa. You can also find the previous occurrence by using a capital F. The following command moves the cursor to the position of the closest previous a in the current line: Fa. A semicolon (;) repeats the last Find command.

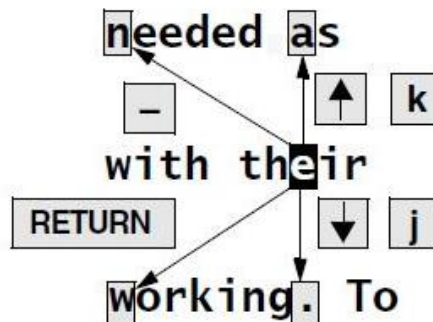
Moving the Cursor by Words



Key	Results
W and w	Moves the cursor forward
B and b	Moves the cursor backwards
E and e	Moves the cursor to the end of the next word

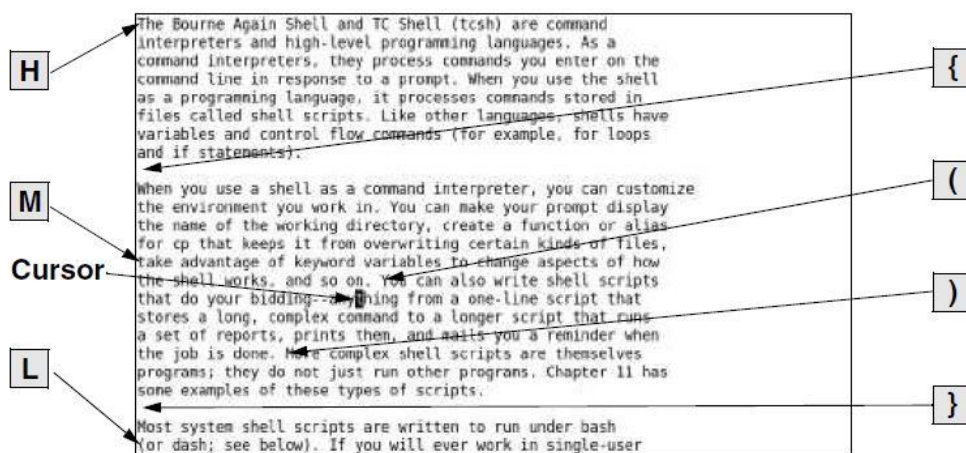
Groups of punctuation count as words. The command 15w moves the cursor to the first character of the fifteenth subsequent word. The W key is like the w key but moves the cursor by blank-delimited words, including punctuation, as it skips forward. The B key moves the cursor backward by blank-delimited words. E moves it to the end of the next blank-delimited word.

Moving the Cursor by Lines



Key	Results
RETURN	To the beginning of next line.
j/ DOWN Arrow	Down one line to the character just below the current character.
k and UP Arrow	Up one line to the character just above the current character.
-	To the end of the next line.
)	Forward to the beginning of the next sentence.
(	Forward to the beginning of the next paragraph.
}	Backward to the beginning of current sentence.
{	Backward to the beginning of current paragraph.

Moving the Cursor Within the Screen



Key	Result
H	To the left end of the top line of the screen
M	To the middle line
L	To the bottom line

Viewing Different Parts of the Work Buffer

The screen displays a portion of the text that is in the Work buffer. You can display the text preceding or following the text on the screen by scrolling the display. You can also display a portion of the Work buffer based on a line number. Press CONTROL-D to scroll the screen down (forward) through the file so that vim displays half a screen of new text. Use CONTROL-U to scroll the screen up (backward) by the same amount. If you precede either of these commands with a number, vim scrolls that number of lines each time you press CONTROL-D or CONTROL-U for the rest of the session (unless you again change the number of lines to scroll). The CONTROL-F (forward) and CONTROL-B (backward) keys display almost a whole screen of new text, leaving a couple of lines from the previous screen for continuity. On many keyboards you can use the PAGE DOWN and PAGE UP keys in place of CONTROL-F and CONTROL-B, respectively.

Deleting and Changing of Text

- Undoing changes
- Deleting characters
- Deleting text
- Changing text
- Replacing text
- Changing case

Undoing Changes

The u command (Undo) restores text that you just deleted or changed by mistake. A single Undo command restores only the most recently deleted text. With the compatible parameter set, vim can

Linux and Shell Scripting

undo only the most recent change. The U command restores the last line you changed to the way it was before you started changing it, even after several changes.

Deleting Characters

The x command deletes the current character. You can precede the x command by a Repeat Factor to delete several characters on the current line, starting with the current character. The X command deletes the character to the left of the cursor.

Deleting Text

The d (Delete) command removes text from the Work buffer. The amount of text that d removes depends on the Repeat Factor and the Unit of Measure. After the text is deleted, vim is still in Command mode.

Command	Result
dl	Deletes current character (same as the x command)
d0	Deletes from the beginning of line
d^	Deletes from first character of line
dw	Deletes to end of word
d3w	Deletes to end of third word
db	Deletes from beginning of word
dw	Deletes to end of blank delimited word
dB	Deletes to beginning of blank delimited word
d7B	Deletes from seventh previous beginning of blank delimited word
d)	Deletes to end of sentence
d4)	Deletes to end of fourth sentence
d{	Deletes from beginning of sentence
d}	Deletes to end of paragraph
d{	Deletes from beginning of paragraph
d7{	Deletes from 7 th paragraph preceding from beginning of paragraph
d/text	Deletes upto next occurrence of word text
dfc	Deletes on current line upto and including next occurrence of character c
dtc	Deletes on current line upto next occurrence of c
D	Deletes to end of line
D\$	Deletes to end of line

Unit 07: The Shell and Popular Editors

dd	Deletes current line
5dd	Deletes five lines starting with current line
dL	Deletes through last line on screen
dH	Deletes from first line on screen
dG	Deletes through end of work buffer
D1G	Deletes from beginning of work buffer

Changing Text

The `c` (Change) command replaces existing text with new text. The new text does not have to occupy the same space as the existing text. You can change a word to several words, a line to several lines, or a paragraph to a single character. The `C` command replaces the text from the cursor position to the end of the line. The `c` command deletes the amount of text specified by the Repeat Factor and the Unit of Measure and puts vim in Input mode. When you finish entering the new text and press ESCAPE, the old word, line, sentence, or paragraph is changed to the new one. Pressing ESCAPE without entering new text deletes the specified text (that is, it replaces the specified text with nothing).

Command	Result
cl	Changes current character
cw	Changes to end of word
c3w	Changes to end of third word
cb	Changes from beginning of word
cW	Changes from end of blank delimited word
cB	Changes from beginning of blank delimited word
C7B	Changes from beginning of seventh previous blank delimited word
C\$	Changes to end of line
c0	Changes from beginning of line
c)	Changes to end of sentence
c4)	Changes to end of fourth sentence
c(	Changes from beginning of sentence
c}	Changes to end of paragraph
c{	Changes from beginning of paragraph
c7{	Changes from beginning of seventh preceding paragraph
ctc	Changes of current line upto next occurrence of c
C	Changes to end of line
cc	Changes current line

Replacing Text

The `s` and `S` (Substitute) commands also replace existing text with new text. The `s` command deletes the current character and puts vim into Input mode. It has the effect of replacing the current character with whatever you type until you press `ESCAPE`. The `S` command does the same thing as the `cc` command: It changes the current line. The `s` command replaces characters only on the current line. If you specify a Repeat Factor before an `s` command and this action would replace more characters than are present on the current line, `s` changes characters only to the end of the line (same as `C`).

Command	Result
<code>s</code>	Substitute one or more characters for current character
<code>S</code>	Substitute one or more characters for current line
<code>5s</code>	Substitute one or more characters for five characters, starting with current character

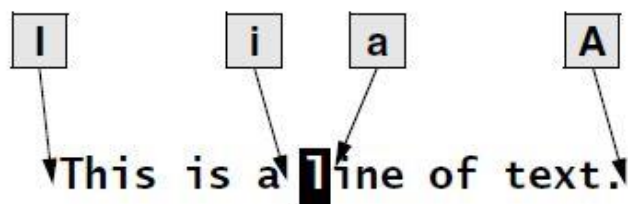
Changing Case

The tilde (`~`) character changes the case of the current character from uppercase to lowercase, or vice versa. You can precede the tilde with a number to specify the number of characters you want the command to affect. For example, the command `5~` transposes the next five characters starting with the character under the cursor but will not transpose characters past the end of the current line.

7.13 Input Mode

The Insert, Append, Open, Change, and Replace commands put vim in Input mode. While vim is in this mode, you can put new text into the Work buffer. To return vim to Command mode when you finish entering text, press the `ESCAPE` key.

Inserting Text



Key	Results
<code>i</code>	Put vim in the insert mode and places the text before the current character.
<code>I</code>	Places the text at the beginning of the current line.
<code>a</code>	Places the text after the current character.
<code>A</code>	Places the text after the current line.
<code>o</code>	Opens a new line below the current line.
<code>O</code>	Opens a new line above the current line.

Replacing Text

The `r` and `R` (Replace) commands cause the new text you enter to overwrite (replace) existing text. The single character you enter following an `r` command overwrites the current character. After you enter that character, vim returns to Command mode—you do not need to press the ESCAPE key. The `R` command causes all subsequent characters to overwrite existing text until you press ESCAPE to return vim to Command mode.

Quoting Special Characters in Input Mode

While you are in Input mode, you can use the Quote command, `CONTROL-V`, to enter any character into the text, including characters that normally have special meaning to vim. Among these characters are `CONTROL-L` (or `CONTROL-R`), which redraws the screen; `CONTROL-W`, which backs the cursor up a word to the left; `CONTROL-M`, which enters a NEWLINE; and `ESCAPE`, which ends Input mode. To insert one of these characters into the text, type `CONTROL-V` followed by the character. `CONTROL-V` quotes the single character that follows it.

7.14 Emacs

Emacs is one of the oldest and most versatile text editors available for Linux and UNIX-based systems. It's been around for a long time (more than twenty years for GNU emacs) and is well known for its powerful and rich editing features. The emacs editor, which is coded in C, contains a complete Lisp interpreter and fully supports the X Window System and mouse interaction. Version 22 has significant internationalization upgrades: an extended UTF-8 internal character set four times bigger than Unicode, along with fonts and keyboard input methods for more than 30 languages. Also, the user interface is moving in the direction of a WYSIWYG (what you see is what you get) word processor, which makes it easier for beginners to use the editor. You never need to switch emacs between Input and Command modes, emacs is a modeless editor.

emacs Versus vim

- 1) Like vim, emacs is a display editor: It displays on the screen the text you are editing and changes the display as you type each command or insert new text.
- 2) Unlike vim, emacs does not require you to keep track of whether you are in Command mode or Insert mode: Commands always use `CONTROL` or other special keys.
- 3) The emacs editor inserts ordinary characters into the text you are editing (as opposed to using ordinary characters as commands), another trait of modeless editing. For many people this approach is convenient and natural.
- 4) As with vim, you use emacs to edit a file in a work area, or buffer, and have the option of writing this buffer back to the file on the disk when you are finished. With emacs, however, you can have many work buffers and switch among them without having to write the buffer out and read it back in.
- 5) Like vim, emacs has a rich, extensive command set for moving about in the buffer and altering text. This command set is not “cast in concrete”—you can change or customize commands at any time.
- 6) Finally, and very unlike vim, emacs allows you to use Lisp to write new commands or override old ones. This feature is called online extensibility,

Getting Started with emacs

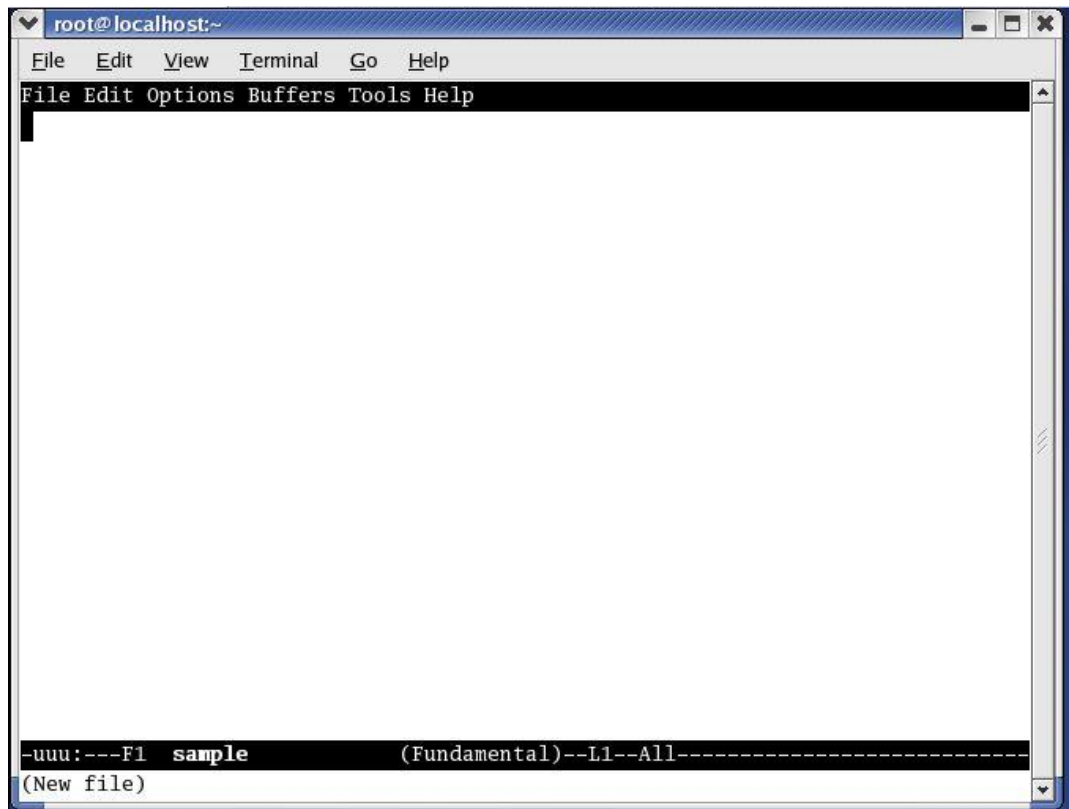
To edit a file named `sample` using emacs as a text-based editor, enter the following command: `$ emacs -nw -q sample`.

- **-nw option:** It must be the first option on the emacs command line, tells emacs not to use its X interface (GUI).

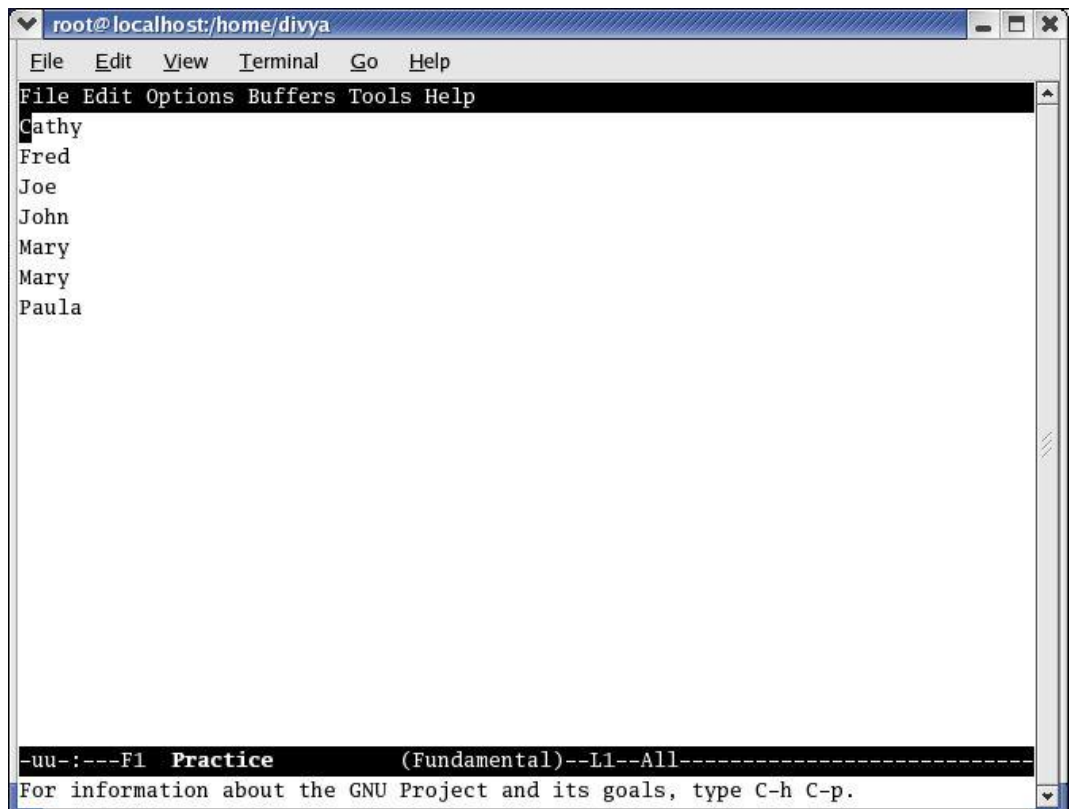
Linux and Shell Scripting

- **-q option:** It tells emacs not to read the ~/.emacs startup file. Not reading this file guarantees that emacs will behave in a standard manner and can be useful for beginners or for other users who want to bypass a .emacs file.

The command `$ emacs -nw -q sample` starts emacs, reads the file named sample into a buffer, and displays its contents on the screen or window. If no file has this name, emacs displays a blank screen with (New File) at the bottom.

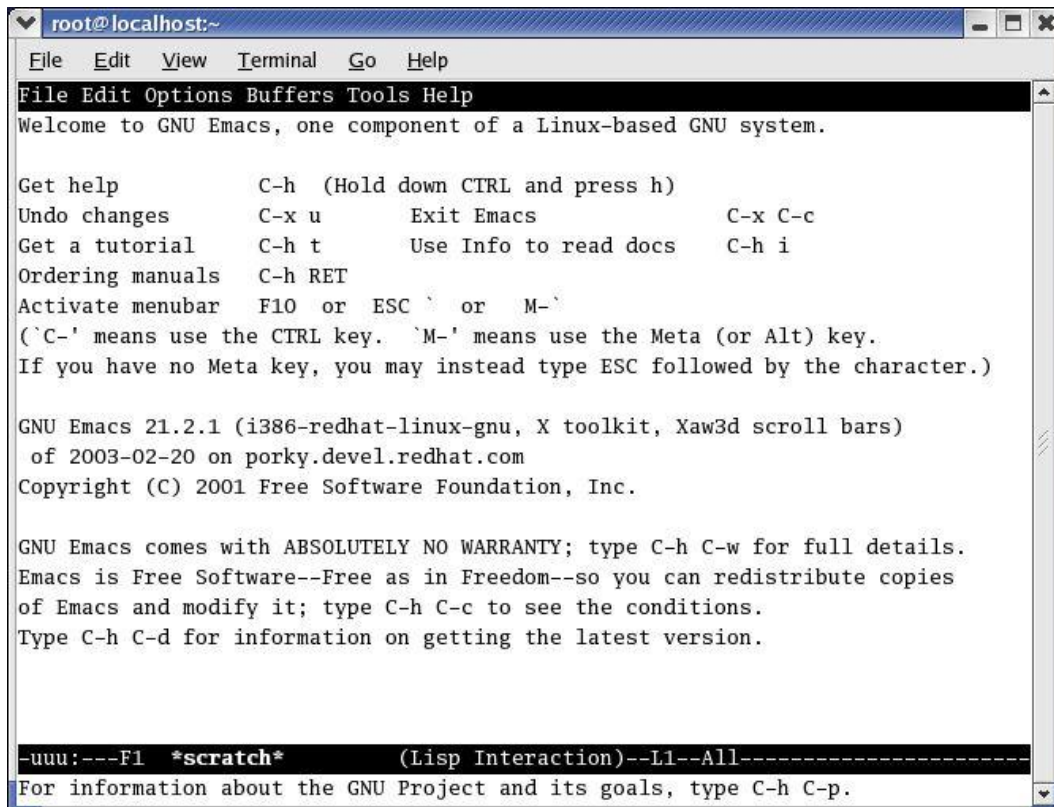


If the file exists, emacs displays the file and a different message.



Unit 07: The Shell and Popular Editors

If you start emacs without naming a file on the command line, it displays a welcome screen that includes usage information and a list of basic commands.



```

root@localhost:~
File Edit View Terminal Go Help
File Edit Options Buffers Tools Help
Welcome to GNU Emacs, one component of a Linux-based GNU system.

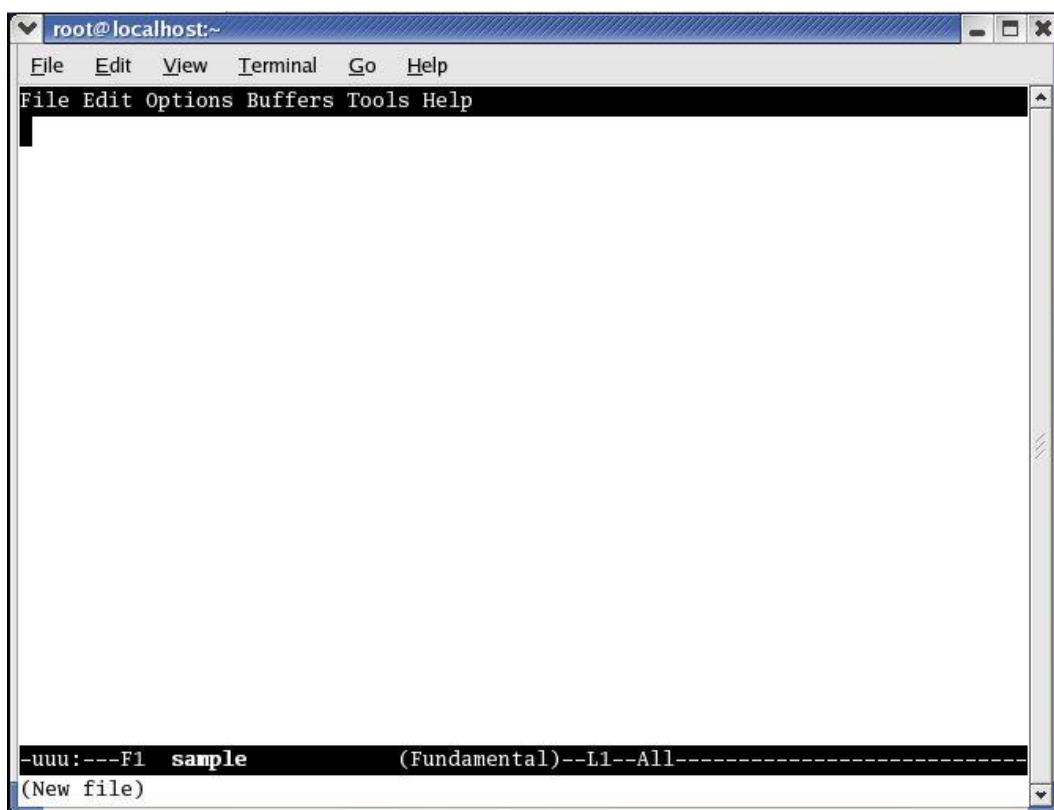
Get help          C-h (Hold down CTRL and press h)
Undo changes      C-x u          Exit Emacs          C-x C-c
Get a tutorial    C-h t          Use Info to read docs C-h i
Ordering manuals C-h RET
Activate menubar  F10 or ESC ` or M-`
('C-' means use the CTRL key. 'M-' means use the Meta (or Alt) key.
If you have no Meta key, you may instead type ESC followed by the character.)

GNU Emacs 21.2.1 (i386-redhat-linux-gnu, X toolkit, Xaw3d scroll bars)
of 2003-02-20 on porky.devel.redhat.com
Copyright (C) 2001 Free Software Foundation, Inc.

GNU Emacs comes with ABSOLUTELY NO WARRANTY; type C-h C-w for full details.
Emacs is Free Software--Free as in Freedom--so you can redistribute copies
of Emacs and modify it; type C-h C-c to see the conditions.
Type C-h C-d for information on getting the latest version.

-uuu:---F1 *scratch* (Lisp Interaction)--L1--All-----
For information about the GNU Project and its goals, type C-h C-p.

```



```

root@localhost:~
File Edit View Terminal Go Help
File Edit Options Buffers Tools Help

-uuu:---F1 sample (Fundamental)--L1--All-----
(New file)

```

Initially, emacs displays a single window. At the top of the window is a reverse-video menubar that you can access using a mouse or keyboard. From the keyboard, F10, META-` (back tick), or META-x tmm-menubar RETURN displays the Menubar Completion List window. At the bottom of the emacs window is a reverse-video titlebar called the Mode Line. At a minimum, the Mode Line

Linux and Shell Scripting

shows which buffer the window is viewing, whether the buffer has been changed, which major and minor modes are in effect, and how far down the buffer the window is positioned. When multiple windows appear on the screen, one Mode Line appears in each window. At the bottom of the screen, emacs leaves a single line open. This Echo Area and Minibuffer line (they coexist on one line) is used for messages and special one-line commands. If you make an error while you are typing in the Minibuffer, emacs displays the error message in the Echo Area. The error message overwrites the command you were typing, but emacs restores the command in a few seconds. A cursor is either in the window or in the Minibuffer. All input and nearly all editing take place at the cursor. As you type ordinary characters, emacs inserts them at the cursor position. If characters are under the cursor or to its right, they are pushed to the right as you type, so no characters are lost.

Exiting

The command to exit from emacs is CONTROL-X CONTROL-C. If you want to cancel a half-typed command or stop a running command before it is done, press CONTROL-G. The emacs editor displays Quit in the Echo Area and waits for another command.

Inserting Text

Typing an ordinary (printing) character pushes the cursor and any characters to the right of the cursor one position to the right and inserts the new character in the space opened by moving the characters.

Deleting Characters

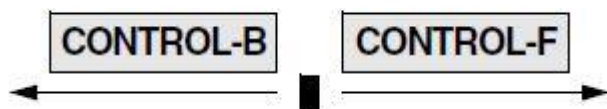
Depending on the keyboard and the emacs startup file, different keys may delete characters in different ways.

- **CONTROL-D** typically deletes the character under the cursor, as do DELETE and DEL.
- **BACKSPACE** typically deletes the character to the left of the cursor.

Moving the Cursor

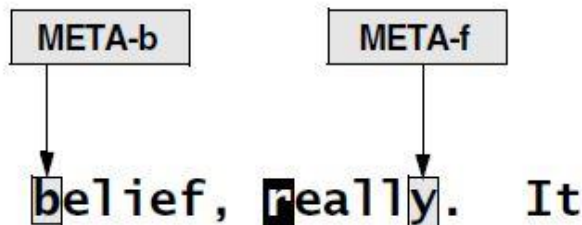
You can position the cursor over any character in the emacs window and move the window, so it displays any portion of the buffer. You can move the cursor forward or backward through the text various textual units – for example, characters, words, sentences, lines, and paragraphs.

Moving the Cursor by Characters



Key	Results
Control - B / Left Arrow Key	Moves the cursor backward one character.
Control - F / Right Arrow Key	Moves the cursor forward one character.

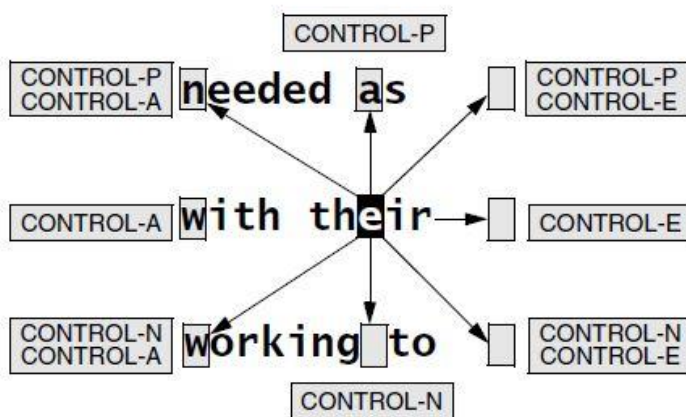
Moving the Cursor by Words



META-f: Moves the cursor forward one word. To invoke this command, hold down the META or ALT key while you press f. If the keyboard you are using does not have either of these keys, press ESCAPE, release it, and then press f.

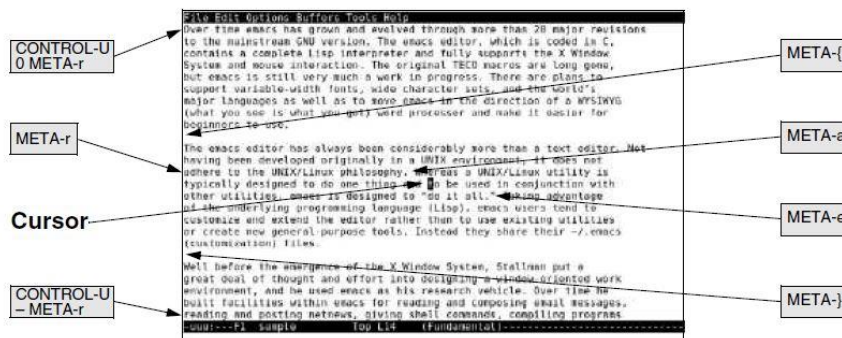
META-b: Moves the cursor backward one word, leaving the cursor on the first letter of the word it started on.

Moving the Cursor by Lines



Key	Result
CONTROL-A/CONTROL-P	Moves the cursor to the beginning of the line it is on
CONTROL-E/CONTROL-P	Moves it to the end
UP ARROW key or CONTROL-P	Moves the cursor up one line to the position directly above where the cursor started
DOWN ARROW key or CONTROL-N	Moves the cursor down one line to the position directly above where the cursor started

Moving the Cursor by Sentences, Paragraphs, and Window Position



Key	Results
META-a	It moves the cursor to the beginning of the sentence the cursor is on
META-e	It moves the cursor to the end of the sentence the cursor is on.
META-{	It moves the cursor to the beginning of the paragraph the cursor is on.
META-}	It moves it to the end of the paragraph the cursor is on.
META-r	It moves the cursor to the beginning of the middle line of the window.
CONTROL-U META-r	It moves the cursor to the beginning of the top line (line zero) in the window.
CONTROL-U- (minus sign)	The command moves the cursor to the beginning of the last line of the window

Editing at the Cursor Position

Entering text requires no commands once you position the cursor in the window at the location you want to enter text. When you type text, emacs displays that text at the position of the cursor. Any text under or to the right of the cursor is pushed to the right. Pressing BACKSPACE removes characters to the left of the cursor. The cursor and the remainder of the text on this line both move to the left each time you press BACKSPACE. To join a line with the line above it, position the cursor on the first character of the second line and press BACKSPACE. Press CONTROL-D to delete the character under the cursor.

Saving and Retrieving the Buffer

No matter what changes you make to a buffer during an emacs session, the associated file does not change until you save the buffer. If you leave emacs without saving the buffer (emacs allows you to do so if you are persistent), the file is not changed and emacs discards the work you did during the session. The command CONTROL-X CONTROL-S saves the current buffer in its associated file. The emacs editor confirms a successful save by displaying an appropriate message in the Echo Area.

The emacs GUI

Full mouse support was introduced in version 19. The emacs editor is still evolving toward full internationalization, accessibility, and a simplified user experience that appeals to the widest possible audience. New features include a colorful menubar and toolbar, mixed text and graphics, tooltips, drag and drop editing, and new GUIs for browsing directories and selecting fonts.

Basic Editing Commands

Command functions in emacs usually involve two or three keys. The most common is the Ctrl key, followed by the Alt or Esc key. In emacs literature, Ctrl is shown in short form as “C”. So if you see something like C-x C-c, it means “press the Ctrl key and x together, then press Ctrl and c”. Similarly, if you see C-h t, it means “press Ctrl and h together, then release both keys and press t”. Alt and Esc keys are referred to as “meta” key in emacs lingo. So if you see a notation like M-x, it means “press Alt/Esc/Option/Edit key and x together”. The Enter key is shown as RET (short for “Return”) and The Esc key is often shown as E.

Marking Text Regions

To mark a text region (like selecting text in popular word processors), follow these steps:

Unit 07: The Shell and Popular Editors

- 1) Move the cursor to the position where you would like the selection to start Press **C-Space** (Ctrl + Space Bar) or **C-@** to set a mark. The mini buffer will show a status message of Mark set.
- 2) Move the cursor to the position where you want the region to end.
- 3) The text will be highlighted up to the point where your cursor is now located.
- 4) If you want to “un-mark” the highlighted text, press C-Space or C-@ twice The mini buffer will show a status message of Mark deactivated.

Cut, Copy & Paste

Once you have the text region marked, you can copy or cut the text and paste it elsewhere. For copying the text, press **E-w**. For cutting the text, press **C-w**. Move your cursor to the position where the text needs to be pasted. Press **C-y** (y stands for “yank” - you are yanking the text from one position to another). The contents will be pasted here.

Deleting Text

For deleting, Backspace and Delete keys work just the way you would expect them to work. For deleting a whole word, move the cursor at the beginning of a word and press **M-d**. For deleting multiple words, hold the meta key down and keep pressing d. Words will start deleting one by one. For deleting a whole line, position the cursor where you want it to be and press **C-k**. This would delete the text right up to the end of the line on screen. For deleting a sentence, press **M-k**

Undo & Redo

Undoing the last operation is simple. Press **C-x u**. You can keep repeating this to go backwards. Another key combination is **C-/** (Ctrl + /) or **C-_** (Ctrl + _). For redoing your last undo, press **C-g**, followed by **C-_** (that's Ctrl + Shift+ Underscore). Another way to do the same thing would be to press C-x C-u again (Undoing the Undo).

Search & Replace

There are two search directions in emacs: forward and backward. In forward search, the word you specify will be searched forward from the current cursor position. For backward search, it's the other way round. Press **C-s** for forward search and press **C-r** for backward search.

For replacing text, follow these steps:

1. Press **M-%** (that's Alt + Shift + %). The mini buffer shows the prompt for the text to be searched (Query replace:).
2. Type the text and press Enter. The mini buffer displays a message like (Query replace <search_word> with:). Type the replacing text and press Enter.
3. For each match emacs finds, it will ask whether you would like to make a replacement (Query replacing <search_word> with <replace_word>: (C-h for help)). You can take any of the following actions: Press y to replace the current match found/ press n to skip to the next match/ press q to exit without any replacements (basically escaping)/ press ! to do a global replace without any prompts. (emacs will show a message like replaced n occurrences)

Left, Right and Centre Alignment

For justifying a selected text region, follow these steps:

1. Create a text region to highlight the text you wish to justify
2. Press **M-x**. The mini buffer will await a response
3. Start typing **set-justification-** and press Tab.
4. You will be given completion options like set-justification-center, set-justification-left, set-justification-right, set-justification-none and set-justification-full

Linux and Shell Scripting

5. Complete the justification command (for example set-justification-right) and press Enter.
6. The selected text will be justified.

Converting Case

Command	Result
M-c (c for Capitalize)	Capitalizing a word after the cursor
M-l (l for Lower case)	Converting a word to lower case
M-u (u for Upper case)	Converting a word to upper case
Block select, then C-x C-u	Converting a paragraph to upper case
Block select, then C-x C-l	Converting a paragraph to lower case

Summary

- Kernel is the innermost part of an operating system; a shell is the outermost one.
- The line that contains the command, including any arguments, is called the command line.
- By default, the shell directs standard output from a command to the screen.
- In addition to standard input and standard output, a running program normally has a place to send error messages: standard error.
- The redirect output symbol (>) instructs the shell to redirect the output of a command to the specified file instead of to the screen.
- The redirect input symbol (<) instructs the shell to redirect a command's input to come from the specified file instead of from the keyboard.
- A filter is a command that processes an input stream of data to produce an output stream of data.
- When you run a command in the foreground, the shell waits for it to finish before displaying another prompt and allowing you to continue. When you run a command in the background, you do not have to wait for the command to finish before running another command.
- You can suspend a foreground job (stop it from running without aborting the job) by pressing the suspend key, usually CONTROL-Z.
- The vim editor is not a text formatting program. It is a sophisticated text editor meant to be used to write code (C, HTML, Java, etc.), short notes, and input to a text formatting system, such as groff or troff.
- If you call vim without specifying a filename on the command line, vim assumes that you are a novice and tells you how to get started.
- There are three modes in vim: command mode, input mode and last line mode.
- To put vim in Input mode, press the i (insert before cursor) key or the a (append after cursor) key.
- In vim to correct text, use dd, dw, or x to remove the incorrect text. Then use i, a, o, or O to insert the correct text.
- There are various features of vim: online help, modes of operation, work buffer, emergency procedures, etc.

Unit 07: The Shell and Popular Editors

- The emacs editor, which is coded in C, contains a complete Lisp interpreter, and fully supports the X Window System and mouse interaction. You never need to switch emacs between Input and Command modes, emacs is a modeless editor.

Keywords

- **Kernel:** Kernel is at the nucleus of a computer. It makes the communication between the hardware and software possible.
- **Shell:** A shell in a Linux operating system takes input from you in the form of commands, processes it, and then gives an output.
- **Token:** On the command line each sequence of nonblank characters is called a token or word.
- **Standard output:** It is a place a program can send information, such as text.
- **Standard input:** It is a place that a program gets information from.
- **Redirection:** The term redirection encompasses the various ways you can cause the shell to alter where standard input of a command comes from and where standard output goes to.
- **noclobber:** The shell provides the noclobber feature that prevents overwriting a file using redirection.
- **Bit bucket:** The /dev/null device is a data sink, commonly referred to as a bit bucket.
- **Pipe:** The shell uses a pipe to connect standard output of one command to standard input of another command.
- **tee utility:** The tee utility copies its standard input both to a file and to standard output.
- **Job:** A job is a series of one or more commands that can be connected by pipes. You can have only one foreground job in a window or on a screen, but you can have many background jobs.
- **? special character:** The question mark (?) is a special character that causes the shell to generate filenames. It matches any single character in the name of an existing file.
- *** special character:** The asterisk (*) performs a function similar to that of the question mark but matches any number of characters, including zero characters, in a filename.
- **Builtin:** A builtin is a utility (also called a command) that is built into a shell.

Self Assessment

1. Which of these is used for erasing a word?
 - A. CTRL-H
 - B. CTRL-W
 - C. CTRL-U
 - D. None of the above
2. Standard _____ is a place that a program gets information from.
 - A. Output
 - B. Input
 - C. Error
 - D. None of the above
3. Which symbol is used for appending the standard output to a file?

- A. >>
 - B. <<
 - C. !!
 - D. <>
4. Which of these components of operating system is the outermost part?
- A. Kernel
 - B. Shell
 - C. CPU
 - D. None of the above
5. The redirection input symbol is
- A. >
 - B. <
 - C. !
 - D. None of the above
6. What is another name of pathname expansion?
- A. Local-ling
 - B. Globing
 - C. Met-forcing
 - D. None of the above
7. A _____ within a bracket defines a range.
- A. Hyphen
 - B. Underscore
 - C. Asterisk
 - D. None of the above
8. How many jobs we can run in foreground?
- A. 0
 - B. 1
 - C. 2
 - D. 3
9. Which of these is a suspend key which can suspend a foreground job?
- A. CTRL-Z
 - B. CTRL-D
 - C. CTRL-C
 - D. CTRL-A
10. Which of these symbols puts the vim in last line mode?
- A. :
 - B. ;

- C. “
- D. ?

11. Which of these modes are available in vim?

- A. Command mode
- B. Input mode
- C. Last line mode
- D. All of the above mentioned

12. Which of these characters delete a character?

- A. x
- B. y
- C. z
- D. None of the above

13. Which of these keys can be used to move the cursor backward by one character?

- A. h
- B. i
- C. j
- D. k

14. For backward search, press

- A. C-s
- B. C-b
- C. C-r
- D. C-c

15. The shortcut for exiting from emacs is

- A. CTRL-X CTRL-C
- B. CTRL-X CTRL-X
- C. CTRL-C CTRL-C
- D. CTRL-C CTRL-X

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. B | 3. A | 4. B | 5. B |
| 6. B | 7. A | 8. B | 9. A | 10. A |
| 11. D | 12. A | 13. A | 14. C | 15. A |

Review Questions:

1. What is an operating system? Describe its main components: shell and kernel in detail.

2. What is a command? What are the rules for writing a command? Explain the components of a command.
3. What are standard input and standard output? Explain redirection and noclobber.
4. Explain pipes and filters.
5. How can we run a command in background? Explain with commands. How to move a job from foreground to background?
6. What is filename generation? Which special characters are used in this? Explain the difference of usage of each character.
7. What is a builtin? How can we list out bash and tschbuiltins?
8. What is vim? How can we use vim? Explain the starting, entering of text, moving of cursor, correction of text and exiting in vim with examples and commands.
9. What are modes in vim? Explain its usage.
10. Explain the various features of vim in detail.
11. What is command mode? How can we move the cursor in this?
12. How can we delete and change text in command mode? Explain with commands.
13. What is emacs? Explain its difference with vim.
14. What are basic editing commands in emacs? Explain.



Further Readings

<https://www.informit.com/articles/article.aspx?p=2854374&seqNum=5>

<https://unix.stackexchange.com/questions/986/what-are-the-pros-and-cons-of-vim-and-emacs>

<https://stackoverflow.com/questions/1430164/differences-between-emacs-and-vim>